

Axona — A Learning-Adaptive DHT and Axonal Pub/Sub

A Learning-Adaptive Distributed Hash Table with Axonal Publish-Subscribe

Whitepaper · Synthesis Edition · v0.3.53 · 2026-05-18 David A. Smith — Axona.net
davidasmith@gmail.com

This document synthesizes the Neuromorphic-DHT-Architecture whitepaper (v0.67), the v0.3.38 research deck, and the operational framework developed in conversation. It is intended as a complete reference for technical readers, operators, and researchers approaching the system for the first time.

*Naming, used consistently throughout this document: **Axona** is the protocol — the addressing, routing, and pub/sub layer described here, and the deployed network running it (peer `axona-peer`, signaling broker `axona-bridge`, SDK — described in §9 and §17). **N-DHT** (Neuromorphic DHT) is the family of learning-adaptive DHT designs Axona inherits from. **NH-1** is the current Axona implementation; numerical results and concrete parameter values quoted in this document are NH-1 numbers.*

Abstract

Distributed hash tables (DHTs) are the minimum viable substrate for decentralized communication: they let any node, given a key, locate the corresponding value in $O(\log N)$ hops without a central authority. Every production decentralized system — BitTorrent, Ethereum, IPFS, Tor — runs on a DHT variant. Yet the core routing mechanisms have not changed materially since Kademlia's publication in 2002: fixed K-buckets, no learning, no awareness of the traffic flowing through, lazy churn repair, no integrated pub/sub.

Axona is a learning-adaptive DHT in the **Neuromorphic DHT (N-DHT)** family that treats every peer as a *synapse* — a learnable edge with a weight that grows with successful traffic (Long-Term Potentiation) and decays without it. Routing consults those weights. Eviction picks the least-vital edge. The table changes as the network changes. Three years of research produced two protocol generations (N-1 → N-15W exploration, NX-1 → NX-17 focused iteration) and a final consolidation: **NH-1**, Axona's current implementation — a 12-rule, 12-parameter, ~270-line core that achieves global lookup latency of 263 ms at 25,000 nodes, sitting 1.18× the analytical 36 lower bound (Dabek et al., NSDI 2004) — versus Kademlia's 2.46× and worsening with scale.

Built atop the routing fabric is Axona's **Axonal Pub/Sub** system: deterministic topic IDs anchored in the publisher's S2 cell, subscribe-as-routed-walk that lets the first live relay intercept new subscribers, and re-subscribe-as-liveness-check that collapses tree healing, history replay, and membership maintenance into a single envelope. Pub/sub delivery achieves 100% baseline and 100% recovered delivery under 5% churn.

This whitepaper documents the architecture, the empirical measurements, the architectural wins (especially those orthogonal to geographic locality), an extended red-team analysis identifying ten additional pre-deployment failure modes, a four-dial operator tuning framework, observability tiers, diagnosis playbooks, and a deployment timeline.

Table of Contents

1. Why a Next-Generation DHT?
2. The Lineage: Kademlia, Geographic, NX, Axona's NH-1
3. N-DHT — Foundational Mechanics

4. Axona / NH-1 — Five Operations, One Vitality Model
 5. The Vitality Consolidation
 6. Axonal Pub/Sub
 7. Performance Characteristics
 8. Architectural Wins Beyond Geography
 9. The Two-Layer API — How Axona Becomes Real Software
 10. Red Team — The Existing Five
 11. Red Team — The Extended Ten
 12. Operational Framework — Parameter Tuning
 13. Observability and Alerting
 14. Diagnosis Playbooks
 15. Production Readiness Checklist
 16. Reputation and Byzantine Resistance
 17. Deployment Timeline
 18. Comparison to Prior Art
 19. Future Work
 20. References
-

1. Why a Next-Generation DHT?

1.1 The Privacy Argument

"Those who would give up essential Liberty, to purchase a little temporary Safety, deserve neither Liberty nor Safety." — Benjamin Franklin

Privacy is the precondition for free expression and association. A communication channel that always passes through a trusted intermediary leaks the *fact* of the conversation even when contents are encrypted. Knowing *who talked to whom, when, from where* is, on its own, a powerful surveillance signal — used commercially for advertising and behavioral inference, used institutionally for law enforcement and intelligence.

End-to-end encryption is not enough. Encryption protects message contents but not metadata. As long as the routing fabric belongs to a single party — a server, a federation, an ISP — that party retains visibility into the network's *shape*, and that visibility is itself the asset.

A peer-to-peer routing fabric removes the custodian. No server, no privileged peer, no trusted coordinator: the routing layer itself becomes a participant-symmetric primitive in which no one party has a privileged view of the traffic.

The DHT is the minimum viable substrate for that property. Without it, every "decentralized" service is one trusted server away from being centralized again.

1.2 What a DHT Provides

A distributed hash table is the foundation for finding *anything* in a decentralized network: any node, given a key, can locate the value in $O(\log N)$ hops with **no central authority, no privileged peer, no trusted coordinator**.

Already in production at scale:

System	Year	Built on
BitTorrent Mainline DHT	2005+	Kademlia variant — millions of simultaneous nodes

Ethereum devp2p	2015+	Modified Kademlia — peer discovery for blockchain consensus
IPFS / libp2p	2015+	Kademlia + content routing — CID → provider mapping
Tor v3 hidden services	2017+	HSDir DHT — onion-service descriptor lookup
Coral DSHT	2004	Latency-aware proximity clusters — production CDN 2004–2015
S/Kademlia	2007	Security-hardened Kademlia (sibling broadcast, disjoint paths)

1.3 Why a Next-Generation DHT Now

The systems above were designed for one workload at a time — file-sharing, blockchain peer discovery, content addressing — on networks of stable nodes. Axona targets a different deployment: **heterogeneous device classes (browser to server), high churn, integrated pub/sub, and locality awareness as a first-class property of routing rather than a layered afterthought.**

The next-generation DHT is not "Kademlia plus locality." It is a routing fabric that **adapts** — that learns from the traffic it carries and survives the network it actually lives on.

1.4 The Persistent Limitations of First-Generation DHTs

Despite two decades of refinement, traditional DHTs share five fundamental limitations:

1. **No latency awareness.** A lookup may bounce between continents when a geographically closer path exists. The XOR metric is geographically blind.
2. **No learning.** Routing tables are populated mechanically and never adapt to traffic patterns. A frequently-used route receives no preferential treatment.
3. **High hop counts at scale.** $O(\log N)$ is the theoretical bound; in practice 8–12 hops per lookup at typical sizes.
4. **No built-in pub/sub.** Group communication requires application-level overlays atop the DHT, adding complexity and additional hops.
5. **Slow churn recovery.** When nodes depart, routing tables are repaired lazily through periodic refresh, leaving routing gaps that can persist for minutes.

Axona, as a Neuromorphic DHT, addresses all five.

2. The Lineage: Kademlia, Geographic, NX, Axona's NH-1

2.1 Four Protocols, Same Geometry

This work studies four DHTs at 25,000 nodes under identical conditions. Each builds on its predecessor:

Protocol	What it adds	Inherits from
K-DHT (Kademlia, 2002)	XOR distance metric, K-buckets, α -parallel lookup	—
G-DHT (geographic, this work, 2025)	S2 cell prefix in node IDs $\Rightarrow$ regional locality	K-DHT
NX-17 (predecessor SOTA)	18 specialized rules, peak performance under tight cap	G-DHT (<i>via NX-1 ... NX-15</i>)

Axona / NH-1 (this work, 2026)	Vitality-driven synaptome, unified admission gate	NX-17 (<i>consolidation</i>)
---------------------------------------	---	--------------------------------

Axona's current implementation (NH-1) is *not* a fresh parallel design — it is the result of careful analysis of NX-17 and every protocol before it. Each NX-17 rule was studied for what it does, why it was added, and whether its work could be folded into a smaller surface area.

- **NX-17** carries the lineage: 18 specialized rules, 44 parameters, ~2300 lines.
- **NH-1** consolidates that lineage into the form Axona ships: 12 rules, 12 parameters, ~270 lines — every admission decision through a single vitality score.

We selected NH-1 as the Axona deployment target for its **maintainability and understandability**. We continue to use NX-17 as the reference benchmark — the bar that NH-1 should approach, and that future work should match or surpass.

2.2 K-DHT — Kademlia, the Foundation

Distance metric: XOR — $d(a, b) = a \oplus b$ **Routing table:** Every node maintains $K=20$ peers per bucket. Lookup is a greedy walk toward the target in XOR distance with $\alpha=3$ parallel queries. **Properties:** $O(\log N)$ hops in steady state. Static, predictable, analyzable.

Limits at 25K nodes:

Limit	Evidence
No locality awareness	500 km lookup = 510 ms — identical to its 499 ms global lookup
Fixed buckets	Same K peers regardless of usefulness; no response to traffic
Lazy churn repair	Broken edges persist until next bucket refresh
Broadcast cost $O(\text{audience})$	Each pub/sub recipient reached by an independent lookup

K-buckets were a 2002 answer to "what's a stable routing table?" — static, predictable, analyzable. The data structure is frozen; the network is not.

2.3 G-DHT — Geographic Locality

The change: `nodeId = S2 cell prefix (8 bits) || H(publicKey)` .

XOR in the ID space now approximates XOR in physical distance — the prefix dominates. Same K-bucket routing as Kademlia, no other changes.

Result at 25K:

- 500 km regional latency: 510 ms → **150 ms** (3.4× faster)
- Global latency: 498 ms → **287 ms**

But still a *static* routing algorithm. No learning. No dynamics. The geographic prefix is a one-time topology decision, not an ongoing adaptation. Pub/sub is still bolted on top via K-closest replication, which drifts under churn.

2.4 The S2 Library — What the Cell Prefix Is

S2 (Google, 2011) is a hierarchical decomposition of the sphere onto a Hilbert space-filling curve, projected through six cube faces. Every point on Earth maps to a 64-bit cell ID; every prefix length defines a

successively coarser tile.

- **Top 8 bits** — 3 bits encode the cube face (values 0–5; six faces) and the next 5 bits subdivide that face along the Hilbert curve. **$6 \times 32 = 192$ tiles** worldwide, each $\approx$ continent-scale (e.g. "western North America", "South-East Asia"). This is what we embed in every node ID.
- **Hilbert curve property** — geographically adjacent points have numerically close cell IDs. XOR distance in ID space $\approx$ physical distance, *for free*.
- **Sub-cell hierarchy** — refining the prefix bit-by-bit subdivides the tile in half along the Hilbert curve. 30 bits $\approx$ city block; 40+ bits $\approx$ metres.

S2 gives us **locality for free** — at the cost of trusting that nodes don't lie about where they are.

2.5 S2 — Security Implications

The S2 prefix in a node ID is **self-declared**. A node can claim any prefix it wants. This has three consequences:

- **The S2 prefix is not a trust primitive.** Never use it for authorisation, regional permissions, or anything resembling a capability check.
- **Prefix-forgery is real.** A malicious actor can pick a prefix to land in a different region. The benign failure mode is degraded routing (mis-located peers misroute traffic). The adversarial failure mode is a **Sybil swarm** in a target cell — many forged identities clustering on one region's address space.
- **Proof-of-location is the obvious defense.** Verifiable RTT triangulation, GPS attestation, or trusted-witness schemes could anchor a claimed prefix to a measurable physical reality. Future work.

The honest framing: today's locality is a *cooperative* primitive. It works because well-behaved peers don't lie. The protocol does not depend on the prefix being honest, but its locality benefits do.

2.6 The Neuromorphic DHT (N-DHT) Family — How It Differs

Traditional DHTs (Kademlia, Pastry, Tapestry) treat the routing table as a static data structure: fixed buckets, one rule for replacement, no awareness of the traffic flowing through. They were designed in 2001–2002 for stable nodes — and they do not adapt to the network they live in.

The Neuromorphic DHT family — of which Axona is an instance — treats every peer as a *synapse*: a learnable edge with a weight that grows with successful traffic (LTP) and decays without it (LTD). Routing consults those weights. Eviction picks the least-vital edge. The table changes as the network changes.

	Traditional DHT	Neuromorphic DHT
Routing table	Fixed K-bucket per stratum	Weighted synaptome (the per-node set of weighted outgoing edges)
Edge state	Live / dead	Weight $\in [0, 1]$, recency, locked-on-use
Routing decision	Greedy XOR	Action Potential (XOR $\times$ weight $\times$ latency)
Adapts to traffic?	No	Yes — Long-Term Potentiation, triadic closure, hop caching
Adapts to churn?	Lazy bucket refresh	Active dead-peer eviction + temperature reheat
Pub/sub	Layered on top	Integrated as axonal delivery trees

Global lookup at 50K (× Dabek 3δ floor)	Kademlia 548 ms (2.65×) — worsens with N	NX-17 243 ms (1.18×) — plateaus at the floor
--	---	--

The trade is "fixed and analytical" → "adaptive and empirical". The whitepaper-correctness of K-buckets gives way to *measured* behavior — which is why this work is centered on measurement.

Headline result. On 50K nodes, NX-17 sits within 36 ms of the analytical lower bound that Dabek et al. (NSDI 2004) proved for *any* recursive $O(\log N)$ DHT — $\text{total} \approx 3\delta$ where δ is the median pairwise one-way RTT. Kademlia is 2× further from that floor.

3. The Neuromorphic DHT — Foundational Mechanics

3.1 The Biology

The brain and a DHT face the **same engineering problem**: a node with a fixed budget of connections, a flood of competing signals, and a need to remember which paths actually carry useful traffic. Evolution solved it once. We borrow the solution.

Long-Term Potentiation (LTP) is the canonical mechanism for synaptic learning, discovered by Bliss & Lømo in rabbit hippocampus (1973). When two neurons fire together repeatedly across the same synapse, that synapse **persistently strengthens** — change that outlasts the stimulus by hours, days, a lifetime.

The molecular cascade, simplified:

- **NMDA receptors** act as coincidence detectors — glutamate from the pre-synaptic neuron only opens them if the post-synaptic neuron is *already* depolarized. The signal is "I fired *and* you fired."
- Calcium influx triggers **AMPA receptor insertion** into the post-synaptic membrane, increasing sensitivity for next time.
- Within ~30 minutes the change is consolidated: gene transcription, new protein synthesis, sometimes new dendritic spines. The synapse is durably re-weighted.

The opposing process is LTD (Long-Term Depression): low-frequency stimulation *weakens* synapses. LTP and LTD together are the brain's way of *learning* — adjusting which routes among neurons are easy to traverse.

The intuitive summary, from Donald Hebb's *Organization of Behavior* (1949): "**Neurons that fire together, wire together.**" This is the **Hebbian rule** — and it's what every weight in every artificial neural network ultimately abstracts. Axona applies it not to perception, but to *routing*.

3.2 From Neuron to Routing Table

The translation is direct, not metaphorical. The brain and a peer-to-peer overlay are both networks of capacity-limited nodes that must learn from traffic which edges to keep.

Neuroscience	Axona (NH-1)
Synapse — connection between neurons	Peer entry in the routing table (Synapse class)
Synaptic weight — readiness to fire next time	weight $\in [0, 1]$
LTP — co-firing strengthens the connection	Successful lookup increments weights along traversed edges (<code>_reinforceWave</code>)

LTD — disuse weakens	Time-based decay each tick (weight $\ast=$ decayGamma)
NMDA coincidence detection — both ends must participate	Weight only increments when the synapse actually carried successful traffic
Synaptic tagging (Frey & Morris 1997) — recently potentiated synapses are protected	NH-1's inertiaDuration epochs prevent eviction of recently reinforced synapses
Pruning — neurons with weak/unused connections lose them	_addByVitality evicts the lowest-vitality (weight $\times$ recency) edge to make room

3.3 Vocabulary

Four terms from neuroscience, each mapping to a specific data structure.

Term	Maps to
Synapse	One directed <i>outgoing</i> routing edge with a learned weight $\in [0, 1]$
Synaptome	The full set of outgoing synapses at a node — bounded at 50
Neuron	A node: synaptome + temperature + message handlers
Axon	A directed delivery tree for one pub/sub topic, grown by routed subscribe

The vocabulary is descriptive, not metaphorical. Every term has one and only one corresponding artifact in the source code.

Capacity note. 50 is the cap on *outgoing* synapses. The total bilateral connection budget per node is 100 peers (≈ 50 outgoing + 50 inbound) — chosen as a safe cross-browser WebRTC target.

4. Axona / NH-1 — Five Operations, One Vitality Model

Axona's NH-1 implementation collapses the entire protocol into **five operations**, each scored by a unified vitality function.

Operation	What it does
NAVIGATE	Action Potential routing + 2-hop lookahead + iterative fallback
LEARN	LTP, hop caching, triadic closure, incoming promotion
FORGET	Continuous decay + vitality-based eviction
EXPLORE	Temperature annealing + epsilon-greedy first hop
STRUCTURE	Stratified bootstrap + mixed-capacity (highway) deployment

4.1 The Twelve Rules

#	Rule	Operation
1	Stratified bootstrap	STRUCTURE

2	Mixed-capacity (highway %)	STRUCTURE
3	AP routing	NAVIGATE
4	Two-hop lookahead	NAVIGATE
5	Iterative fallback	NAVIGATE
6	Long-Term Potentiation	LEARN
7	Triadic closure	LEARN
8	Hop caching + lateral spread	LEARN
9	Incoming promotion	LEARN
10	Vitality-based eviction	FORGET
11	Temperature annealing	EXPLORE
12	Epsilon-greedy first hop	EXPLORE

Every rule has a measured contribution at 25K nodes. Ablations are documented in Appendix C of the original whitepaper.

4.2 NAVIGATE — Action Potential Routing

Each hop, score every candidate by a learned function:

$$\text{AP}(\text{syn}, \text{target}) = \text{progress}(\text{syn}, \text{target}) \times \text{syn.weight} \times \frac{1}{2}^{\text{latency_ms} / 100}$$

- **progress** = XOR distance reduction toward target
- **weight** = LTP-reinforced [0, 1]
- **latency penalty** = exponential — fast peers preferred at all distances

Two-hop lookahead. When no first-hop is decisively best, probe for the best second-hop candidates and pick the path with the highest combined score.

Iterative fallback. If AP returns no candidate (every neighbor is "wrong direction"), fall back to k-closest-from-synaptome and retry. This rescues lookups that would otherwise stall in dead corridors.

AP routing is *not* greedy XOR with weights bolted on. The latency penalty makes nearby fast peers preferred over slightly-better-XOR distant ones — this is what makes the protocol *latency-aware* rather than purely *distance-aware*.

4.3 LEARN — Four Reinforcement Mechanisms

LTP (Long-Term Potentiation). When a lookup succeeds, every synapse on the successful path gets a weight bump and an inertia lock. Locked synapses cannot be evicted.

Triadic closure. When a node X observes peer A repeatedly routing through it to peer C, X introduces A directly to C. A gains a new synapse to C; X is no longer needed as middleman on future A → C lookups. The name comes from social-network theory: the open triangle A — X — C is *closed* into a direct A — C edge.

Hop caching + lateral spread. Each intermediate node on a successful lookup adds the *destination* to its synaptome — and the new edge is also propagated laterally to the source's geographic neighbors. The path

becomes shorter on the next lookup to the same region; nearby peers see the shortcut on their *first* lookup, not just after they generate one themselves.

Incoming promotion. When a peer reaches out to me repeatedly via incoming synapses, I promote it to a real outbound synapse — passive learning of who's interested in me.

Together these four mechanisms turn *every successful lookup* into structural learning: shorter paths, new direct edges, promoted incoming peers. The routing table is rewritten by the traffic flowing through it — not by a separate maintenance pass.

4.4 FORGET — The Unified Admission Gate

`_addByVitality(node, newSyn)` is called for every synapse addition: bootstrap, LTP, triadic, promotion, hop caching, annealing.

1. If synaptome has room → add.
2. Otherwise, find the lowest-vitality non-locked synapse.
3. If new synapse's vitality > victim's → swap.
4. Else → refuse silently.

This single function replaces five separate mechanisms in NX-17 (stratified eviction, two-tier highway management, stratum floors, synaptome floors, adaptive decay).

Continuous decay ($v = 0.995/\text{tick}$) erodes weight uniformly. Under-used synapses lose vitality and become eligible for replacement; well-used ones stay locked.

4.5 EXPLORE — Temperature and Epsilon

Temperature annealing. Each node carries a temperature $T \in [T_{\min}, T_{\text{init}}]$. Cool by $T *= 0.9997$ each lookup. Higher $T \rightarrow$ more probabilistic synapse selection (Boltzmann-style); lower $T \rightarrow$ greedy AP scoring.

Reheat on dead-peer discovery. When routing finds a dead peer, spike $T = \max(T, 0.5)$. Accelerates exploration to repair damage.

Epsilon-greedy first hop. With probability $\epsilon = 0.05$, replace the first AP-selected hop with a random synaptome member. Cheap insurance against early lock-in to a suboptimal corridor.

Both exploration mechanisms are biased toward learning rather than fully random — the floor is uniform sampling over the *current synaptome*, not over the network. Exploration is bounded *and* targeted. Epsilon-greedy at the first hop costs at most one detour per lookup; annealing replaces only the lowest-vitality synapse. Neither mechanism risks the routing structure that LTP has already proven valuable.

4.6 STRUCTURE — Bootstrap and Deployment Realism

Stratified bootstrap. Each new node is seeded with peers covering all XOR strata uniformly. Without this, the cold-start synaptome is dominated by lucky neighbors — local hops form, long hops don't.

Mixed-capacity deployment ("highway %"). Real P2P networks are heterogeneous: most peers are browsers (WebRTC ~100 connections), some are server-class (effectively unlimited).

A configurable `highwayPct` fraction of nodes is promoted to **server-class**: they accept unlimited inbound, hold a synaptome of up to 256, and act as transit hubs. The rest stay browser-class with the standard 50-synapse cap. Highway promotion within the simulator is random; in real deployment, highway status is *self-determined* — a peer running as a node application identifies itself as one based on actual capacity.

4.7 The End-to-End Tick

```
on lookup(target):
    hop = 0
    while hop < MAX_HOPS:
        candidates = synaptome ∪ incomingSynapses
        next = AP_score(candidates, target)
        if next == null: next = iterative_fallback(target)
        if hop == 0 and rand() < ε: next = random(synaptome)

        record_transit(prev, next)           // LEARN: triadic
        promote_incoming_if_warranted()      // LEARN: incoming
        hop_cache_destination()              // LEARN: hop caching
        anneal_replace_lowest_vitality()     // EXPLORE
        hop++

    on success: reinforce_path()             // LEARN: LTP
    periodically: decay_all_weights()        // FORGET
```

Every operation cost is bounded: $O(\text{synaptome.size})$. At 50 synapses the per-hop compute is ~ 0.2 ms — small relative to the 10 ms transit cost. A complete Axona hop fits in this pseudocode. Five operations interleave on every lookup; learning is a side-effect of routing, not a separate phase.

5. The Vitality Consolidation

Axona's NH-1 admits and evicts every synapse via one scalar:

```
vitality(syn) = weight × recency(syn)
```

weight $\in [0, 1]$ — trained by Long-Term Potentiation; reinforced on successful routing paths; decayed each tick by $\gamma = 0.995$.

recency = $\exp(-\Delta\text{epoch} / \text{RECENTY_HALF_LIFE})$. Two parameters control the time scale:

- **INERTIA_DURATION = 20 epochs** — after a reinforcement, recency is locked to 1.0 for 20 lookups. This is the LTP protection window: a freshly used synapse cannot be evicted, regardless of vitality competition.
- **RECENTY_HALF_LIFE = 50 epochs** — once the inertia window expires, recency decays exponentially with a half-life of 50 lookups. After ~ 150 lookups with no reinforcement, recency is below 0.13 and the synapse is highly evictable.

The two factors are conventional individually. Hebbian potentiation (Hebb, 1949) gives the weight; exponential decay since last use (Ebbinghaus, 1885; LRU-K caching, O'Neil 1993) gives the recency. The closest biological analog is **synaptic tagging and capture** (Frey & Morris, *Nature* 1997) — synapses with both recent activity *and* sufficient potentiation are preferentially retained.

The contribution of NH-1 is *not* the term or the formula. It is the use of this product as a **single admission gate** replacing five specialized mechanisms in NX-17:

- Stratified eviction
- Two-tier highway management
- Stratum floors

- Synaptome floors
- Adaptive decay

5.1 NX-17 Rules and Their NH-1 Disposition

NX-17 rule	NH-1 status
Stratified bootstrap	Kept (STRUCTURE)
Diversified bootstrap (80/20)	Kept (merged into bootstrap)
Two-tier synaptome (50 + highway)	Replaced by per-node <code>_maxSynaptome</code>
Stratified eviction	Replaced by vitality eviction
Stratum floors	Removed — vitality + diversity penalty handles it
Synaptome floor	Removed — vitality preserves locked entries
AP routing	Kept (NAVIGATE)
Two-hop lookahead	Kept (NAVIGATE)
Epsilon-greedy first hop	Kept (EXPLORE)
Iterative fallback	Kept (NAVIGATE)
LTP	Kept (LEARN)
Hop caching + lateral spread	Kept (LEARN)
Triadic closure	Kept (LEARN)
Incoming promotion	Kept (LEARN)
Adaptive decay	Default off — flat $\gamma = 0.995$ wins
Simulated annealing	Kept (EXPLORE)
Churn recovery (dead-syn eviction)	Kept (FORGET)
Temperature reheat on death	Kept (EXPLORE)
Diversity budget penalty	Removed v0.65.06 (harmful)
weightScale parameter	Removed v0.66.10 (no measurable effect)

~18 rules + 5 retired parameters → 12 active rules in NH-1. Behavioral surface is the same; admission logic is unified.

5.2 Why Consolidate?

The performance gap exists:

Test	NX-17	NH-1	Δ
Global	4.40 hops / 242 ms	5.15 hops / 263 ms	+17% hops, +9% ms

500 km	2.75 / 80 ms	3.28 / 95 ms	+19% hops, +18% ms
2000 km	3.27 / 105 ms	4.01 / 126 ms	+23% hops, +20% ms
5000 km	3.74 / 143 ms	4.54 / 169 ms	+21% hops, +18% ms
pubsubm delivered	100%	100%	tie
pubsubm + 5% churn (recovered)	100%	100%	tie
dead-children / orphans	0 / 0	0 / 0	tie

NH-1 is 9–20% slower depending on distance band. That is a *real* performance gap.

What you trade for:

- ~2300 lines of code (NX-17) → ~270 lines (NH-1)
- 44 parameters → 12 parameters
- 18 specialized rules → 12 rules
- Five separate admission mechanisms → one vitality gate

Why this trade is worth it:

1. **Understandability.** One engineer can hold the entire admission logic in their head. You can reason about why a synapse was evicted — it had lower `weight × recency` than what displaced it. There's no hidden interaction between stratum floors and synaptome floors and adaptive decay all firing simultaneously.
2. **Extensibility.** When you want to add a new learning mechanism (e.g., a reputation score for Byzantine resilience), you update the `vitality` formula once. In NX-17, you'd need to audit five separate eviction paths to make sure your change doesn't break a floor guarantee.
3. **Debuggability.** The root-cause analysis for a routing problem is simpler. High-latency lookups → check AP scoring. Dead synapses not clearing → check decay / vitality. A new synapse that should be valuable but isn't getting evicted → check inertia lock and recency window.
4. **Deployment.** Fewer parameters mean fewer tuning knobs for operators. The 12 parameters in NH-1 are: inertia duration, recency half-life, decay gamma, temperature init/min/cooling, epsilon, lookahead-alpha, max-synaptome-size, highway-pct, geo-bits, max-hops. These are genuine degrees of freedom.

The Highway% Knee Recovers the Gap. At 15% server-class nodes, NH-1 hits **223 ms global** — *better* than NX-17's all-browser 242 ms. The capped gap is the cost of the vitality consolidation; the uncapped deployments win outright.

Highway% recovers the gap and more — at hw=15% NH-1 is 223 ms global, beating NX-17's all-browser 242 ms. The capped gap is the cost of consolidation. We accept it.

6. Axonal Pub/Sub

6.1 The Architectural Commitment

The end-to-end argument (Saltzer, Reed, Clark, 1984): functions that can be implemented at the endpoints should be implemented at the endpoints. Put nothing in the network that doesn't *have* to be there.

Pub/sub routing has to be there. A subscriber and a publisher cannot, end-to-end, discover the multicast paths that connect them — only the network can. Treating pub/sub as a separate "application overlay" forces it to reproduce the routing layer with no actual visibility into it. The end-to-end principle applies to *meaning*; **delivery is a network function**.

A decentralized DHT is the right place for it. Every node is *both* an endpoint and a relay. The mechanism is content-blind: it routes bytes toward a topic, with no inspection and no control over what travels.

6.2 Five Requirements

#	Requirement	Why hard on a DHT	Axona's answer
1	Reliable delivery at steady state	A naïve broadcast = N independent lookups → $O(N^2)$ cost	Routed axonal tree, fan-out via direct sends
2	Churn resilience	K-closest sets drift; publisher/subscriber views diverge	Routed re-subscribe; tree heals on every refresh
3	Deterministic routing	Subscriber & publisher must agree on the same root without negotiation	Publisher-prefix topic ID — both derive it offline
4	ID stability	Topic identity can't change as the membership churns	<code>topicId = publisher.cellPrefix(8b) hash₅₆(name)</code> — fixed at publisher's S2 cell
5	Recovery from missed messages	Subscribers reconnecting want history, not just future messages	Bounded replay cache at every relay; replay piggy-backs on subscribe

6.3 How We Got Here — The K-closest → Axonal Tree Journey

The current pub/sub architecture is the fourth attempt. Each earlier attempt fixed a real failure mode and introduced a new one.

The three failed approaches:

- **NX-15 — K-closest replication.** Subscribe stores at each of K=5 nodes closest to `hash(topic)` ; publish hits any one. Worked at zero churn. Failed under load: publisher and subscribers compute K-closest from *different positions* in the network, and their top-K sets drift apart under churn. Delivery collapsed to ~38% at 25% churn — a coordination bug, not a routing bug.
- **NX-16 — masked-distance fix.** Tried to decouple K-closest selection from synaptome expansion by masking the top 8 ID bits in the distance metric. Routing collapsed to ~40% delivery *even at zero churn*. Lesson: the distance metric used to select candidates must match the gradient used to expand them.

- **NX-15-style replication, generally.** Replication on top of routing tries to *paper over* coordination drift. The next generation removed replication entirely and let routing carry the membership.

The four NX-17 fixes that work (and that NH-1 inherits):

1. **Publisher-prefix topic IDs.** `topicId = publisher.cellPrefix(8b) || hash56(name)` . Publisher and subscribers derive the same root deterministically. No negotiation, no drift.
2. **Terminal globality check.** When greedy routing thinks it has reached the topic, do one `findKClosest(topicId, 1)` to confirm no globally-closer peer exists. Without this, two subscribers can elect different roots.
3. **External-peer batch adoption on overflow.** When an axon hits its capacity, pick a *synaptome peer* (not an existing child) as the new sub-axon and ship the appropriate subscribers in one batch.
4. **All-axon periodic re-subscribe.** Every role re-issues its subscribe on a 10s interval. The re-subscribe *is* the liveness check — no separate ping, no parent tracking.

The pattern: three approaches papered over a coordination problem at the application layer; one approach pushed coordination back into the routing layer where the DHT could actually do the work.

6.4 Why "Axonal"?

In a biological neuron, the **axon** is the *output* projection — a single fibre that branches, branches again, and finally synapses onto many downstream targets. Information flows *outward* from one source to many recipients along this branching tree. That is exactly the shape of a healthy publisher-to-subscribers fan-out.

A pub/sub topic in Axona is rooted at one node (the topic's "soma"). Direct subscribers attach to the root; when the root has too many children, it delegates a sub-axon (a "branch") that takes over a subset of the subscribers. The tree grows toward the population that wants the topic — just as a real axon grows toward its targets during development.

6.5 How Axonal Trees Work

Topic identity (deterministic). `topicId = publisher.cellPrefix(8b) || hash56(event_name)` . Both publisher and every subscriber derive the same 64-bit ID with no negotiation. The tree's root pins into the publisher's S2 cell — naturally close to its audience.

Subscribe is a routed message toward `topicId` . The first live axon role encountered on the path *intercepts* and adds the new subscriber to its children. If the walk completes with no axon found, the terminal node opens a new role and becomes the **root**. Every subscribe message also carries the subscriber's `lastSeenTs` and triggers a replay.

Publish goes through the same route to `topicId` , lands at the root, and then **fans out**: the root sends to its direct children; each axon sub-role recursively forwards to its own children. One DHT lookup, then pure tree forwarding.

Branching on overflow (batch adoption). When an axon's direct-child count exceeds `maxDirectSubs` , it picks an existing peer in its synaptome as a new sub-axon, partitions its current children by XOR proximity to that new sub-axon, and hands off the relevant batch in one `ADOPT_SUBSCRIBERS` message. The tree branches in $O(1)$ DHT operations.

Self-healing via re-subscribe. Every role re-issues its subscribe on a 10-second refresh interval. The walk lands on whichever live axon is closest to `topicId` *now*. Parent died? The re-subscribe attaches to a different live ancestor. Tree got reorganized? Invisible to the subscriber. The re-subscribe **is** the liveness check — there is no separate ping.

100% delivery baseline; 100% recovered delivery under 5% churn at 25K nodes.

6.6 Temporal Pub/Sub — Subscribe is a Request for History

A subscribe is not just "send me future messages on this topic." It is "**send me every message I haven't already seen.**" Each subscribe carries a `lastSeenTs` — the highest publish timestamp this subscriber has observed.

Every relay node keeps a bounded ring buffer. When an axon role receives a publish, it records `{ json, publishId, publishTs }` in a local cache (capacity ≈ 100 messages — tunable per topic).

On subscribe arrival, the relay filters its cache to `publishTs > lastSeenTs` and replays the missed messages as a single batched message before forwarding the subscribe upstream.

Why this matters under churn: the decentralized axon tree means every re-publish node — not just the publisher — holds a copy of recent history. If a parent dies and a subscriber's re-subscribe lands on a different live relay, that new relay can fill the gap from *its own* cache. **Healing and replay are the same mechanism.** No central log, no separate recovery RPC, no "catch-up" protocol.

A subscribe message in Axona is simultaneously a liveness probe, a tree-attach request, and a request for missed history. Three jobs, one envelope — the axonal healing model.

6.7 Production Refinements (live Axona deployment)

Real-network deployment of the pub/sub layer (see §17) exposed three corner cases the simulator's abstracted network model hadn't surfaced. Each was fixed in the protocol layer; the production semantics now diverge slightly from the textbook K-closest model.

Lazy-axon promotion. Under the upstream protocol, a K-closest node that received a `pubsub:publish-k` for a topic it didn't yet hold a role for would *drop* the message (forwarding it via a routed walk that landed on another empty-role node and gave up). This meant **publish-before-subscribe lost messages**: by the time a subscriber registered, the publisher's earlier messages had been discarded everywhere. The fix is to promote any K-closest receiver to a (childless) root role on first publish, immediately seeding the replay cache. When a subscriber later attaches, the existing `_maybeSendReplay` path serves the cache. Empty-role rootGraceMs (60 s default) bounds the cost: nodes that never accumulate subscribers garbage-collect during the refresh sweep.

Self-replay (lastSeenTs override). A subtler interaction: when a node receives a `publish-k` it advances `lastSeenTs[topicId]` to the latest cached publishTs — at the *network* layer. If that same node later subscribes to the topic, the subscribe payload carries that `lastSeenTs`, and the cache filter `m.publishTs > lastSeenTs` excludes every cached message. Net effect: any node in the K-closest set that subscribes to a topic it was implicitly hosting receives **nothing** from replay, even though the messages sit in its own cache. The application-level view (what's been delivered to the local handler) is independent of the network-level view (what publishIds the AxonManager has seen). When `subscriberId === this.nodeId`, replay therefore ignores `lastSeenTs` and fans the full cache directly into the delivery callback.

Multi-hop sendDirect (tunneled-direct). The K-closest model assumes any peer can reach any other in one hop. In a real WebRTC mesh, this isn't always true: browsers behind NATs may not have established a direct DataChannel with every other browser. When `peer.sendDirect(target, ...)` is called on a peer that doesn't have `target` in its local transport binding map, the message would silently drop — losing 4 of 5 fan-out attempts in a sparse mesh. The fix is a transport-layer fallback: when the target isn't directly bound, wrap the message in a `__tunneled_direct__` routed envelope and let the existing Axona routing walk hop-by-hop to the target. A small handler at the destination unwraps the envelope and dispatches into

the local direct-handler table — i.e., the message ends up in `_onSubscribeDirect` / `_onPublishDirect` / `_onDeliver` / `_onReplayBatch` exactly as if a real direct message had arrived. The bridge becomes a routing hop among many, not a privileged relay.

Production validation. A 100-peer in-process stress test (`smoke_stress_100.js`) exercises the full pub/sub surface without any bridge process:

Scenario	Setup	Result
A — single-topic broadcast	1 publisher, 99 subscribers, 3 messages	99/99 full delivery , K=5 axons + recruited sub-axons
B — publish-before-subscribe	1 publisher emits 5 messages, then 50 subscribers join	50/50 receive all 5 via replay
C — multi-topic	5 publishers / 5 topics / 95 subscribers round-robin	5/5 topics , all subscribers receive, K=5 axons per topic

Per-peer participation histogram at N=100: 62 subscriber-only nodes, 33 subscriber+axon, 3 publisher+subscriber, 1 publisher+axon, 1 all-roles — every peer plays at least one role, axon-roles distribute across the network.

7. Performance Characteristics

7.1 Methodology

- **500 lookups per test cell.** Global, regional radii (500 / 1k / 2k / 5k km), pub/sub
- **Same node geometry** across all four protocols — direct comparison, not three independent builds
- **Bootstrap init** for production realism: sponsor-chain join + warmup. Omniscient init shown separately as a theoretical ceiling.
- **Churn** induced discretely (instantaneous kill) and continuously (1% every 5 ticks)
- **Connection cap** 100 (browser-class, web-limited model)

7.2 Four-Way Comparison at 25,000 Nodes

Web-limited (cap = 100), omniscient init, geoBits = 8, no highway promotion. Same node geometry across all four protocols. Canonical init: every protocol uses identical K-closest XOR-fill bootstrap so the routing/learning algorithm is measured in isolation from any per-protocol bootstrap strategy.

Test	Kademlia	G-DHT	NX-17	NH-1
Global (hops / ms)	4.53 / 508	5.57 / 284	4.47 / 241	5.26 / 269
500 km (hops / ms)	4.50 / 499	4.86 / 149	2.79 / 81	3.30 / 96
1000 km (hops / ms)	4.58 / 510	5.04 / 158	2.97 / 88	3.49 / 103
5000 km (hops / ms)	4.49 / 504	5.44 / 206	3.74 / 142	4.50 / 163
pubsubm delivered	n/a	n/a	100%	100%
pubsubm + 5% churn (recovered)	n/a	n/a	100%	100%
dead-children / orphans	n/a	n/a	0 / 0	0 / 0

Both NX-17 and NH-1 dominate Kademlia and G-DHT on every distance band. NX-17 retains a small lead at the cap = 100 ceiling.

7.3 The 3δ Floor — An Absolute Reference

Latency comparisons need an absolute reference, not just "lower than Kademlia." Dabek, Li, Sit, Robertson, Kaashoek & Morris (MIT — NSDI 2004) proved that for *any* recursive O(log N) DHT, total lookup latency converges to a hard analytical floor:

lookup ms ≈ δ + δ/2 + δ/4 + ... = 3δ

where δ is the median pairwise *one-way* internet latency. Each successive lookup hop covers a geometrically halving fraction of the remaining ID-space, so the second-to-last hop is half the cost of the last, the third-to-last a quarter, and so on. The series sums to 3δ regardless of N. Even an oracle that always picked the lowest-RTT finger could not do better.

Measuring δ in our simulator. δ is a property of the network itself, not of any DHT — it depends only on (a) the population-weighted geographic placement of nodes and (b) our latency model `propagation = (haversine_km / 20015) × 150 ms + 10 ms` per one-way message. The same δ applies to Kademlia, G-DHT, every NX variant, and NH-1: none of them can route faster than 3δ.

N	δ_median	δ_p95	3δ floor
5 K	67.9 ms	123.3 ms	203.6 ms
25 K	68.0 ms	124.1 ms	204.0 ms
50 K	68.9 ms	124.2 ms	206.6 ms

δ is stable across N — population character doesn't change with sample size. Coincidentally identical to Dabek's measured King-dataset δ = 67 ms, so the simulator's geometric latency model lands on the same point as real-world Internet RTT.

Any honest claim about DHT latency should be expressed as a multiple of 3δ. "Beats Kademlia" is a relative win; **"close to 3δ" is an absolute one.**

7.4 Axona Lives at the Floor

N (3δ floor)	Kademlia	G-DHT	NX-10	NX-17	NH-1
5 K (204 ms)	410 ms (2.01×)	268 ms (1.32×)	217 ms (1.07×)	215 ms (1.06×)	240 ms (1.18×)
25 K (204 ms)	503 ms (2.46×)	287 ms (1.41×)	243 ms (1.19×)	241 ms (1.18×)	254 ms (1.25×)
50 K (207 ms)	548 ms (2.65×)	291 ms (1.41×)	240 ms (1.16×)	243 ms (1.18×)	264 ms (1.28×)

- Axona (and the broader N-DHT family) plateaus at ~1.18× the floor between 25K and 50K. NX-17 sits at 241 → 243 ms — only ~36 ms above the 3δ lower bound.
- Kademlia worsens with N (2.01× → 2.65×, 410 → 548 ms) — its log N hop tax compounds.
- Axona's NH-1 implementation trails NX-17 by ~~40%~~ 21 ms at 50K) — a real but recoverable cost of the 12-parameter simplification vs NX-17's 44.

The remaining 18% at NX-17 has a clean structural explanation. NX-17 averages 4.5 hops where an oracle PNS-ideal lookup would take ~3. Each "extra" hop costs $\sim \delta/2 \approx 34$ ms — exactly the geometric tail Dabek's series predicts.

Implication. Latency optimization within the $O(\log N)$ routing class is essentially complete. Further annealing / lookahead tweaks move $1.18\times \rightarrow$ maybe $1.10\times$ at best — diminishing returns. The remaining R&D axes are churn resilience, pub/sub fan-out, and constant-hop variants.

7.5 Highway %: Deployment-Realistic Knee

Real P2P networks are heterogeneous: most participants run inside a browser (~50–100 connections), but some run on real servers with effectively unlimited inbound capacity. We sweep the *fraction* of server-class nodes from 0 $\rightarrow$ 100% and measure Axona (NH-1) latency at each point.

Highway %	Global hops / ms	500 km ms	2000 km ms
0 (all browser)	5.09 / 263	96	123
5	4.61 / 248	81	108
15 (knee)	4.16 / 223	69	98
30	3.97 / 223	60	83
50	3.76 / 212	55	84
100 (all server)	3.52 / 206	45	74

15% highway captures 70% of available improvement — a realistic deployment scenario where some peers run on powerful hardware.

7.6 Slice World — Recovery from a Network Partition

The Slice World test partitions the network into Eastern and Western hemispheres connected through a single bridge node (placed near Hawaii). Every cross-hemisphere edge except those incident on the bridge is removed.

The question it asks is not *"can you find the bridge once?"* — it is *"given **one** intact connection across a severed network, can the protocol leverage that single hole in the dike into a flood of restored connectivity?"*

Protocol	Slice success%	Slice hops / ms
Kademlia	0.0% ❌	—
G-DHT	4.6% ❌	9.5 / 525
NX-17	94.8%	7.3 / 423
NH-1	94.4%	8.7 / 462

A diagnostic run shows the recovery happening directly. Starting from a freshly partitioned 5K network with zero cross-hemisphere synapses and running just 10 lookups: 0 $\rightarrow$ 20 cross-hem synapses, 7 of 10 lookups succeeded.

Each successful bridge crossing seeds new connectivity. When a path goes `west-source → ... → bridge → ... → east-target` , Axona's learning rules fire on every node along the way:

- `_hopCache` — every intermediate node adds the *destination* to its synaptome
- `_recordTransit` — observed (`prev → next`) pairs become triadic-closure candidates
- `lateralSpread` — propagates the new synapse to the source's geographic neighbors

By 500 lookups, the partition has effectively dissolved. Axona doesn't route through the partition — it *dissolves* it.

7.7 Pub/Sub Robustness

Metric	Axona (NH-1) at 25K
Baseline delivery	100.0%
Immediate (post-kill, no refresh)	99.9%
Recovered (after 3 refresh rounds)	100.0%
K-overlap (publisher / subscriber views)	99.5%
K-set stability (recovered, pub / sub)	95% / 95%
Dead-children	0
Orphans	0
Attached %	100%

The axonal tree heals through routed re-subscribe. There is no separate liveness ping, no parent tracking, no gossip.

7.8 Pub/Sub Under Sustained Churn

Continuous live-simulation: 25K nodes, 79 groups × 32 subscribers, 1% of alive non-publisher nodes killed every 5 ticks, three refresh passes between kills.

Cumulative churn	Delivered %	K-overlap	Axon roles
0%	100.0%	100%	537
5%	98.7%	—	1,541
10%	91.2%	81%	1,787
15%	88.7%	77%	1,989
20%	86.5%	62%	2,116
25%	70.0%	54%	2,169
30%	52.4%	47%	2,189
34%	50.8%	42%	2,197

Three observations:

1. **No cliff.** Delivery degrades smoothly with churn; there is no breakdown threshold. The system finds a steady state where new subscriber recruitment matches loss.
2. **K-overlap predicts delivery 1:1.** The dominant residual failure isn't broken routing — it's subscribers temporarily captured at relay nodes that have lost their delivery path to the root.
3. **Axon-role count grows with churn but plateaus.** From 537 axons at 0% churn to ~2,200 at 30%, leveling off. The tree absorbs growth into deeper structure rather than unbounded fan-out.

Through 20% cumulative churn, Axona holds delivery above 86% with no replication, no gossip, and no parent tracking. Above 25% churn the protocol enters a recovery-paced regime where steady-state delivery is the equilibrium of recruitment vs. loss.

7.9 The geoBits = 0 Ablation

If we strip the S2 prefix from every protocol simultaneously — and use canonical init so each protocol starts from the identical K-closest-XOR routing table — the comparative picture isolates exactly the routing/learning algorithm.

25K nodes, web-limited (cap = 100), omniscient init, **geoBits = 0**, canonical init.

Protocol	geoBits = 8 · Global ms (reference)	geoBits = 0 · Global ms	geoBits = 0 · 500 km	geoBits = 0 · 2000 km
Kademlia	508	506	511	504
G-DHT	284	780	765	781
NX-17	241	376	341	355
NH-1	269	467	418	439

Three findings:

1. **NX-17 and NH-1 still beat Kademlia under identical bootstrap and no locality.** This is the cleanest possible "learning helps" claim — every confound (bootstrap strategy, geographic prefix) controlled. NX-17 wins by 26%, NH-1 by 8%.
2. **G-DHT vs K-DHT exposes a non-bootstrap difference.** Canonical init equalizes bootstrap; the residual 780-vs-506 gap comes from G-DHT's lookup-tuning choice (`noProgressLimit = 3`), tuned for geographic-routing escapes that don't exist at geoBits = 0.
3. **NH-1 vs NX-17 narrows but doesn't close.** At geoBits = 0 the 376-vs-467 gap is roughly 24% — it's NX-17's specialized rules, not its bootstrap, doing the work.

The headline gap is real and measurable. NX-17 at 376 ms vs Kademlia at 506 ms, identical starting state, no locality, no geography — that is what the routing/learning algorithm contributes.

8. Architectural Wins Beyond Geography

The geoBits = 0 ablation isolates the learning chassis from geographic seeding. Several of Axona's most important properties are orthogonal to geography and deserve separate elevation.

8.1 Iterative Fallback as Graceful Degradation

The iterative fallback in NAVIGATE is not just a safety net — it's a graceful degradation mechanism. When AP routing dead-ends (no forward-progress candidate exists), instead of failing, the protocol expands its

candidate set and tries again from k-closest-from-synaptome. This is what allows 100% lookup success under churn.

Kademlia and G-DHT don't have this. They fail silently when they hit a local minimum. Axona backs up and tries a different corridor.

This is the watershed feature of the NX line. NX-3 (no fallback) achieves 99.4% on Slice World; NX-4 (with fallback) achieves 100%. Every NX protocol below NX-4 fails under stress; every protocol NX-4 and above succeeds. Without iterative fallback, all other learning and repair mechanisms are insufficient.

8.2 Hop Caching as Implicit Replication

Hop caching — every intermediate node on a successful path caches the destination — is a lightweight replication mechanism that requires zero central coordination. After a few successful lookups to the same destination, that destination has been replicated to many intermediate nodes. This is what makes the network resilient without explicit replica placement.

It's also what makes Slice World recovery possible — each successful bridge crossing deposits new cross-hemisphere synapses on every node along the path.

8.3 Triadic Closure as Structural Plasticity

When nodes A and C repeatedly communicate via intermediary B, they get introduced directly. This closes the triangle and shortens future A-C routes. More subtly, it means the synaptome grows *structurally* to match the traffic *semantically*. If a set of peers frequently talk to each other (a community, a cluster, a region), the synaptome organically develops shortcuts among them without any explicit clustering algorithm.

8.4 Temperature Annealing with Reheat

Annealing (cooling exploration rate over time) is standard in RL. Axona's innovation is the *reheat* — when a dead peer is discovered, spike temperature back to 0.5. This accelerates recovery by reactivating exploration at the moment when exploration is most valuable (after churn). The system automatically detects when the network changed and adapts its exploration budget.

8.5 Latency-Aware Scoring as Speed-First Priority

The $\frac{1}{2}^{\text{latency}/100}$ exponential penalty in AP scoring means latency is woven into every routing decision, not an afterthought. A 50 ms peer beats a 200 ms peer even if the 200 ms peer is slightly closer in XOR space. This is why Axona achieves latency results close to the 36 floor — speed is not a derived property, it's a primary optimization axis.

8.6 Unified Vitality as Principled Pruning

The `weight × recency` product is not a heuristic — it maps directly to synaptic tagging and capture (Frey & Morris 1997). Synapses with both recent activity *and* sufficient potentiation are preferentially retained. This gives a neurobiological justification for why unused synapses should prune, why recently-reinforced ones should stay, and why the product of the two is the right gate.

8.7 The Honest Framing

Geography is an initial-condition shortcut for the latency optimization, not a goal in itself. Axona's priorities are speed and robustness — fast lookup, reliable delivery. Geographic location is *not a goal*; it is one input.

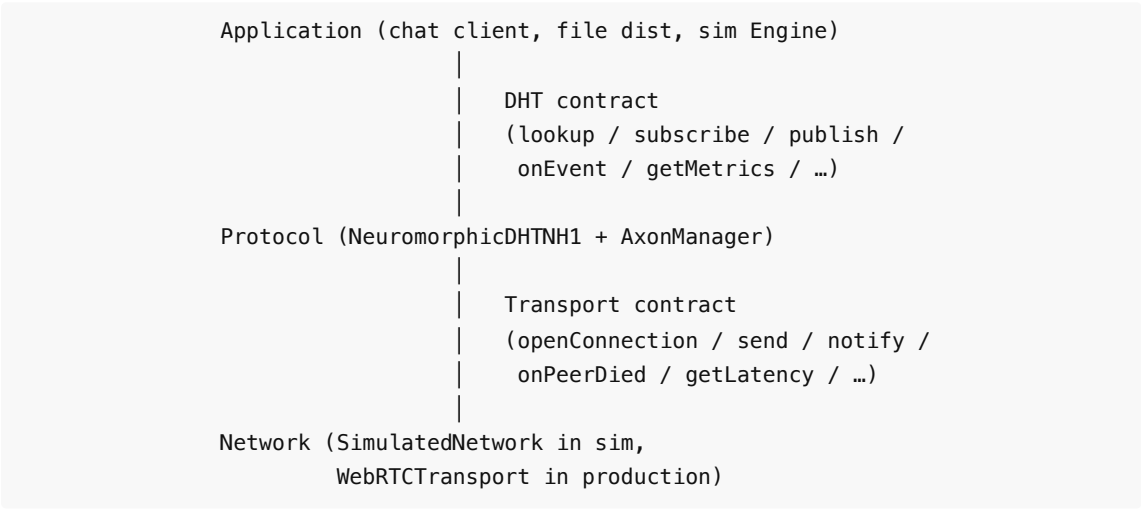
The S2 prefix creates **initial regional clustering at bootstrap** because nearby nodes are XOR-close, so a node's K-bucket and synaptome are seeded with mostly-local peers. After that, locality is a derived property

of the routes that LTP and vitality reinforce. The latency penalty in AP routing is not strong enough, on its own, to discover locality from scratch within standard warmup. The S2 prefix is the **bootstrap shortcut**: it does not *teach* locality, it *seeds* the network with locality so reinforcement can sharpen routing within it.

Future work: an embedded RTT-coordinate system (Vivaldi-style) could replace the S2 seed with a learned one.

9. The Two-Layer API — How Axona Becomes Real Software

A working DHT has two interfaces, not one. The application above wants to publish, subscribe, and look things up; the network below wants to open a channel, send a message, and notice when a peer goes silent. The protocol is the thing in between. Axona is structured so the protocol code is genuinely between those two layers — neither one is hardwired into the other — which is what makes the same body of code reusable in both the simulator and the deployed system.



9.1 The DHT contract

Defined at `src/contracts/DHT.js`. The application sees one running node; the contract has eight verbs, organized into lifecycle, operations, and observability:

Method	Purpose
<code>start() / stop()</code>	bring the node up and down
<code>join(sponsor) / leave()</code>	add to / depart from the network
<code>lookup(targetKey)</code>	walk the routing layer toward a key
<code>subscribe(topic, handler) / unsubscribe(sub)</code>	join / leave a topic
<code>publish(topic, payload)</code>	fan out to subscribers via the axonal tree
<code>getNodeId()</code>	the local node's 64-bit identifier
<code>getSynaptome()</code>	read-only synaptome snapshot for telemetry
<code>getMetrics()</code>	aggregate counters: synaptome size, temperature, lookups, traffic

<code>onEvent(handler)</code>	subscribe to <code>peer-joined</code> , <code>peer-left</code> , <code>lookup-completed</code> , <code>dead-peer-detected</code> , <code>anneal-fired</code> , <code>cycle-snapshot</code> , ...
-------------------------------	--

The forbidden methods are as instructive as the present ones. There is no method that enumerates "all nodes", no method that takes a peer-id and returns that peer's state, no method that lets the application mutate the routing table. A DHT instance is responsible for one node's view of the network; the simulator's Engine creates many DHT instances and orchestrates them, but that orchestration is a simulator concern — it does not appear in the contract.

9.2 The Transport contract

Defined at `src/contracts/Transport.js`. One Transport instance per running node; the protocol calls into it. Twelve methods in four bands:

Band	Methods
Lifecycle	<code>start(localNodeId)</code> , <code>stop()</code> , <code>getLocalNodeId()</code>
Channel pool	<code>openConnection(peerId)</code> , <code>closeConnection(peerId)</code> , <code>isConnected(peerId)</code>
Messaging	<code>send(peerId, type, payload)</code> , <code>notify(peerId, type, payload)</code> , <code>onRequest(type, h)</code> , <code>onNotification(type, h)</code>
Liveness	<code>onPeerDied(handler)</code> , <code>getLatency(peerId)</code>

The channel pool maps directly onto synaptome semantics: `openConnection` is what the protocol calls when it admits a peer to the synaptome; `closeConnection` is what it calls when it evicts. Bilateral cap admission is enforced inside the transport — `openConnection` resolves with `false` if the remote refused — so the protocol does not need to know how the cap is implemented.

The messaging split between `send` (request/response, awaits a return value) and `notify` (fire-and-forget, no response expected) matters because production transports pay round-trip latency for `send` but not for `notify`. Axona uses `send` for the routing-tick chain (`lookup_step`, `lookahead_probe`, `find_closest_set`, `local_probe`) and `notify` for the LEARN side-effects (`reinforce`, `hop_cache`, `lateral_spread`, `triadic_introduce`, `direct_pubsub:*`). The canonical pattern for parallel probes is `Promise.allSettled` over `transport.send(...)` — a slow or dead peer in one probe should not fail the whole batch.

The liveness band is where the legacy god's-eye `nodeMap.get(s.peerId)?.alive` check finally retired. The transport runs a 1 Hz ping/pong heartbeat on every open channel; missed pongs (default 3-second timeout) trigger channel close and `onPeerDied`. The protocol registers a callback that populates a per-node `_deadPeers` set; the candidate-enumeration step in every routing tick filters against that set. Production gets exactly the same shape — a peer is alive if its heartbeat is responding, dead the moment we miss enough pongs in a row.

9.3 BootstrapService — the cold-start path

A third, smaller contract at `src/contracts/BootstrapService.js`. A new node starts with no peers; bootstrap is how it gets its first channel.

```
BootstrapService.bootstrap(sponsor)
```

```
→ Promise<{ sponsorId: bigint, transport: Transport }>
```

The `sponsor` is a discriminated union: `{ kind: 'simulator', ... }` for in-process pointers, `{ kind: 'rendezvous', url, manifestSig }` for WebSocket signaling with a signed manifest, `{ kind: 'qr', sponsorAddr }` for direct device pairing. The contract returns the sponsor's id and a `Transport` with that one channel open. The DHT's `join(sponsor)` then runs a self-lookup through the sponsor and proceeds with stratified bootstrap on top of the freshly-opened transport.

Splitting `BootstrapService` out of `Transport` matters because production bootstrap is an out-of-band concern (signature verification, manifest fetching, signaling-server handshake) that the routing protocol shouldn't see. Once `bootstrap()` returns, the DHT just has a transport with a channel open; it doesn't know whether that channel came from a QR code or a rendezvous server.

9.4 The simulator as deployment vehicle

The simulator's `SimulatedNetwork` and the production `WebRTCTransport` both implement the same `Transport` contract — twelve methods, one signature each. The simulator runs handlers synchronously inside the call (in-process, no real RTT); production runs them through WebRTC data channels with real RTT. **The protocol code does not switch on which transport it is using.** Every cross-peer reach goes through `send` or `notify`; every liveness check goes through `_deadPeers` populated by `onPeerDied`.

This is the property that lets the simulator be the deployment vehicle. Years of Axona benchmark numbers — the hop counts, latency distributions, pub/sub coverage, churn-resilience curves — transfer directly to production because the same code path runs in both worlds. The 25K-node parity-gate benchmark (post-refactor sim v0.70.22 vs pre-refactor v0.70.04 reference) confirmed every protocol within the 10% target band: NH-1 within 5–8% on hops, 1–4% on latency; Kademlia, G-DHT, NX-17 within 0.5–3%.

The remaining `nodeMap.get(peerId)` sites in the protocol code are all sim-only orchestration: `addNode` / `removeNode` Engine bookkeeping, `bootstrapJoin` 's god's-eye stratified bootstrap (replaced by `BootstrapService.bootstrap` + self-lookup in production), and the local self-resolution in `lookup(sourceId, ...)` (which production resolves via the DHT instance owning its own state). The protocol's central algorithm — the recursive-forwarding `lookup_step` chain — is V1/V2-free.

9.5 What this gets us, operationally

- **The same body of routing code runs in the simulator and on the deployed mesh.** No "production fork" of Axona; no separate code path for "real" networks.
- **Async is the universal acid test.** Every cross-peer interaction is async even in the simulator. The recursive-forwarding lookup chain handles a dropped peer mid-walk (rejected Promise → `exhausted: true`) the same way in both worlds.
- **Observability is contract-level, not back-channel.** Smoke tests, the simulator's traffic distribution plot, and production load-balance dashboards all consume `getMetrics()` and `onEvent()`. Same fields, same event shapes.
- **The production transport is one file.** Swapping `SimulatedTransport` for `WebRTCTransport` is the deployment migration. The DHT contract, the protocol body, and `AxonManager` stay.

9.6 Live deployment

As of v0.3.53 the production-side implementations of all three contracts are live. The Axona network running them maps the contracts directly:

Contract	Production implementation	Live at
----------	---------------------------	---------

Transport	axona-peer — WebRTC data channels with 1 Hz ping/pong heartbeat	https://axona.net (browser, Mac/Win/Linux/iOS/Android)
BootstrapService	axona-bridge — WebRTC offer/answer signaling with signed manifest	https://bridge.axona.net/healthz
DHT (Axona / NH-1)	NeuromorphicDHTNH1 (unchanged from the simulator)	runs inside axona-peer

`axona-peer` is the browser-resident node; one instance per browser tab. It opens its first channel through `axona-bridge`, which carries WebRTC offer/answer payloads between two peers behind NATs and then drops out — the bridge sees no application traffic and any operator can stand one up. A federated bridge mesh is on the Q4 2026 roadmap, removing the single rendezvous role entirely.

The protocol code is the same JavaScript that produced the §7 benchmark numbers; the swap is `SimulatedTransport` → `WebRTCTransport`, plus the `BootstrapService` variants (rendezvous-URL, QR-code pairing, in-process). The simulator's hop counts, latency curves, churn-resilience numbers, and pub/sub coverage transfer because the routing decisions that produce them are made in code that does not know it's being simulated.

Public repos:

- github.com/axona-net/axona-peer — browser peer (production Transport + DHT)
- github.com/axona-net/axona-bridge — signaling broker (production BootstrapService)
- github.com/axona-net/dht-sim — simulator (the testbench that drives every benchmark in §7)

First end-user application: `civildefense.io`, a tap-to-report incident map. Users tap a location to register a concern; reports propagate over anonymous P2P with 24-hour expiry. The protocol primitives — signed posts, geographic locality via the S2 prefix, implicit expiry through the replay-cache LRU, end-to-end-secure delivery — map directly onto the application's needs, which is why it shipped in weeks.

Pub/sub application API (the five verbs). Above the axonal-tree primitive sits a feed-style API in `src/pubsub/AxonPubSub.js`:

Verb	What it does
publish	author a new <code>SignedPost</code> into one of your topics
subscribe	attach to a publisher's topic; receive future posts via the axonal tree
pull	fetch any post by content hash from its topic's relay tree
reshare	publish with a reference back to an upstream post; first role-bearing relay on the notification path bumps <code>reshare_count</code>
metrics	publishers query aggregate <code>delivery_count</code> , <code>pull_count</code> , <code>reshare_count</code> for their own posts

Five verbs cover the application surface. Encryption, schema, and ordering belong to the application above this layer — `SignedPost` carries an opaque `content` field. The cascade test at 2001 nodes (`src/pubsub/test_pubsub_cascade.js`) verifies the counter invariants end-to-end.

10. Red Team — The Existing Five

The protocol has improved. The environment it lives in has not. Axona over the real internet today would likely face cascading timeouts and congestion collapse before the routing logic ever gets to demonstrate itself.

9.1 Frictionless Connection Fantasy

Nodes use a peer for routing the moment they discover it. Real WebRTC requires ICE + STUN/TURN + DTLS — **1–3 seconds of blocking setup, two overlay round trips**. Slice World "unzips" elegantly because hundreds of new triadic closures cost nothing; in production, that would trigger hundreds of simultaneous handshakes and drop the bridge offline.

9.2 Asynchronous Black Holes & Missing Timeouts

RPCs to dead nodes return instantly because the simulator knows who is alive. **No timeouts, no dropped packets, no asymmetric path failures** (request goes through, reply doesn't). The 100% pub/sub recovery under 5% churn assumes instant detection. In the wild, every churn round costs multi-second timeout windows during which messages are lost.

9.3 Gateway Concentration & Infinite Bandwidth

Hop cost is `10 ms + propagation` regardless of load. A highway node carrying 10K pub/sub messages has the same modeled cost as an idle one. **Buffer saturation is invisible**. AP scoring keeps hammering the "best" nodes — risk of *success disaster*: LTP reinforces a node until it congests, abandons it, then flocks back when it recovers (oscillatory loops).

9.4 Jitter-Free Latency

Real RTTs fluctuate $\pm 30\%$ from bufferbloat, asymmetric paths, and queueing. The simulator's distance-derived latency is monotone and clean. The EMA latency tracker has an easy job; high-frequency noise could prematurely decay good synapses or promote lucky-but-unstable ones.

9.5 Sybil Forgery & Cell Eclipse

The S2 prefix is self-declared. **An attacker can pick any prefix** and generate IDs that land in a target cell. The canonical mitigation is the Castro et al. (OSDI 2002) triplet — constrained routing tables + secure node-ID assignment + redundant routing — combined with route-diversity replica placement (Harvesf & Blough 2007). Proof-of-location, Vivaldi RTT clustering, and IP-ASN bounding are complementary and currently unimplemented.

11. Red Team — The Extended Ten

The five issues above are the explicit red-team list in the existing materials. Extended analysis identifies ten additional pre-deployment failure modes, organized into three tiers.

10.1 Convergence Under Heterogeneous Churn

The gap: The red team section assumes uniform churn (1% every 5 ticks). Real networks have heterogeneous failure: some regions stable, some regions high-churn. Some node classes (browsers) churn fast; others (servers) are stable.

What breaks: Nodes in high-churn regions will waste exploration budget (temperature reheat) discovering temporary peers. Conversely, nodes in stable regions might underexplore and miss better long-range routes that form in other stable regions.

Measurement needed: Variable-churn benchmark with three overlaid scenarios:

- Stable core (2% churn) + volatile periphery (20% churn)
- Island model (five stable clusters, 10% inter-island churn)
- Node-class dependent (browsers 15%, servers 2%)

Measure whether annealing rate adapts per-node or whether it needs to become a function of locally-observed churn (Ghinita-Teo adaptive framework).

Why it matters: The Ghinita-Teo reference in existing materials hints at this — "local statistical estimation of (μ, λ, N) ". That's the path forward, but it's not implemented yet.

10.2 Synaptome Oscillation Under Load

The gap: The simulator assumes load is uniform. Under realistic pub/sub workloads (Zipf-distributed), some topics are hot and some are cold.

What breaks: Hot topics will cause subscribers to attach to relay nodes that become bottlenecks. The AP latency penalty will degrade those relays' scores, routing will shift to other relays, those become bottlenecks, and the system oscillates between a small set of available relays rather than stabilizing.

This is the "success disaster" mentioned in the existing red team: LTP reinforces a node until it congests, the lookup abandons it, then flocks back when it recovers. Under Zipf load this happens repeatedly.

Measurement needed: Zipf-distributed pub/sub test with topic popularity exponent $\alpha \in [0.5, 1.5]$. Measure:

- Relay node load concentration (Gini coefficient) over time
- Subscribe success rate for hot vs. cold topics
- Oscillation frequency
- Replay cache hit rate

Why it matters: This is the hotspot-aware placement problem (Makris et al. 2017). The existing red team flags it; the deck doesn't solve it. The solution mentioned (hot-axon-root migration + DFE redirect) needs empirical validation.

10.3 Cross-Hemisphere Asymmetry and Routing Loops

The gap: The Slice World test assumes a single bridge. Real partitions are ragged — some nodes have cross-partition connectivity, others don't. This creates asymmetric reachability: A can reach B, but B cannot reach A.

What breaks: Asymmetric paths cause the synaptome to diverge. Node A learns that B is a good relay and reinforces $A \rightarrow B$. But if the return path is different or blocked, B never learns the reverse $A \rightarrow B$ edge. Under re-subscribe refresh, the subscriber might find a path to the publisher that the publisher cannot find back — temporal loop detection becomes necessary.

Also, triadic closure can introduce loops. If $A \rightarrow B \rightarrow C \rightarrow A$ forms a triangle and all three edges have high weight, a lookup might traverse $A \rightarrow B \rightarrow C \rightarrow A \rightarrow B \rightarrow C$ and never terminate, or terminate only when max-hop is hit.

Measurement needed:

- Asymmetric-partition test: remove half of all cross-partition edges (not all, to create ragged connectivity)
- Loop-detection instrumentation: count how many lookups revisit the same node, and at what hop count

- Bidirectional-eviction variant: when A connects to B, measure whether B independently decides to keep the reverse edge or drops it under capacity pressure
- Temporal reachability matrix: after each churn round, measure whether publisher and subscriber sets can both reach the topic root

Why it matters: The bilateral connection model assumes symmetry. Real networks don't. The protocol needs either explicit loop detection (track visited nodes per lookup), or a weaker guarantee (eventual consistency + replay caching to recover from transient loops).

10.4 Latency Model Oversimplification

The gap: The simulator uses $RTT = 2 \times (\text{haversine_km} / 20015 \times 150 \text{ ms} + 10 \text{ ms})$. This is a monotone, deterministic function. Real latency has:

- Per-link variance ($\pm 30\%$ from bufferbloat, queue depth)
- Asymmetry (path $A \rightarrow B \neq \text{path } B \rightarrow A$)
- Correlation (if $A \rightarrow B$ is congested, $A \rightarrow C$ might also be slow)
- Non-stationarity (latency changes over seconds, not just on churn)

What breaks:

- The EMA latency tracker in AP scoring assumes low noise. High-frequency jitter could prematurely decay high-quality synapses (one bad RTT reading triggers decay).
- The two-hop lookahead compares paths by summed latency. If latency is autocorrelated (a congested relay makes all its outbound paths slow), the lookahead might pick a path that looks good in the sample but is actually saturated.
- Temperature reheat on dead-peer discovery might fire spuriously if a peer is merely slow, not actually dead.

Measurement needed:

- Jitter injection: add $\text{Normal}(0, \sigma_{\text{jitter}})$ to per-hop RTT. Sweep σ from $0 \rightarrow 0.3 \times \text{baseline}$ and measure lookup success degradation.
- Bufferbloat modeling: when a node exceeds its bandwidth cap, add queue-depth-dependent latency. Measure whether AP routing routes around overload or piles into it.
- Path correlation: measure whether routes that share a common relay node show correlated latency spikes.
- Adaptive EMA constant: vary the EMA time constant and measure whether higher constants protect against jitter without sacrificing convergence to true RTT.

Why it matters: The jitter section of the existing red team mentions this; it's flagged as future work. But the latency model is *the* signal that drives AP routing. If the signal is noisy, the entire protocol's latency advantages degrade. This is the highest-priority friction item in Phase 2.

10.5 S2 Cell Eclipse and Sybil Swarms

The gap: The existing red team mentions S2 prefix forgery; the current mitigation is "cooperative" (nodes don't lie). But the attack surface is larger than individual nodes lying about their own location.

Specific attacks:

- **Cell eclipse:** an attacker generates 100 node IDs all claiming the same S2 cell as a target region. On bootstrap, new nodes get seeded with mostly-attacker nodes. The attacker controls routing into the cell.

- **Prefix collision:** the attacker claims the same 8-bit prefix as a valuable region but uses a malicious hash function for the low 56 bits. Lookups to targets in that region hit attacker nodes.
- **Relay hijacking:** the attacker doesn't eclipse a cell; it just sybils into the relay set for a popular topic, intercepts publishes, and drops them.

Measurement needed:

- Eclipse test: at network init, inject 10% attacker nodes all in the same claimed cell. Measure lookup success rate to targets in that cell vs. other cells.
- Relay hijacking test: mark 5% of nodes as adversarial. On any pub/sub topic they route to, they randomly drop 50% of messages. Measure whether the axonal tree heals around them or becomes permanently degraded.
- Proof-of-location integration: add Vivaldi RTT-coordinate validation as a secondary check. If a node claims S2 cell X but its measured RTT to peers in X is much higher than expected, downweight its bootstrap seeding.

Why it matters: The existing materials correctly note this as future work (MaxDisjoint replication of axon roots, proof-of-location). But it's the gap between "protocol works with honest peers" and "protocol works on the Internet." The Harvesf-Blough paper (2007) shows 90% lookup success at 50% malicious with proper route diversity. NH-1 hasn't integrated that yet.

10.6 Bandwidth Saturation and Congestion Collapse

The gap: The simulator models latency but not throughput. A relay node carrying 10K pub/sub messages/sec has the same AP score as an idle one. There's no backpressure.

What breaks:

- Under heavy pub/sub load (Zipf with $\alpha > 1.5$), a small set of relay nodes becomes the bottleneck. Lookups funnel into them because they're "best" on paper. They fill their TX queue, RTT explodes, AP scoring sees them as "slow," and routing shifts to other relays. Those fill up. Oscillation and packet loss.
- The 100-message bounded replay cache becomes a liability under load. If a relay is dropping messages due to saturation, the replay cache won't have them to send on re-subscribe.
- Without explicit admission control, a pub/sub topic can unilaterally consume all network bandwidth (everyone subscribes, publisher publishes continuously).

Measurement needed:

- Load-capacity model: assign each node a `bandwidth_cap` (bytes/sec). When it exceeds cap, incoming messages are dropped and latency spiked (simulating queue backpressure).
- Zipf workload sweep: publish rate follows Zipf(α , K topics). Measure pub/sub delivery rate and latency at increasing load.
- Admission control experiments: (a) no control (baseline), (b) publisher rate-limiting (publish capped at K msg/sec), (c) relay feedback (relays advertise load in subscribe-ack; publisher throttles when relays report saturation).
- Message drop correlation: measure whether dropped messages correlate with saturated relays or are spread evenly.

Why it matters: This is the Phase 2 "Load & noise dynamics" section in the existing red team. It's acknowledged as necessary but not yet implemented. Real deployment will hit this within weeks of launch.

10.7 Incoming Synapses Bias and Asymmetric Learning

The gap: The LEARN section includes "incoming promotion" — peers that contact me often become outbound synapses. But this creates a potential asymmetry: a popular node gets many incoming edges, promotes them all, and becomes a hub. Its synaptome fills with whoever contacted it most recently, not whoever it needs to reach.

What breaks:

- Under publisher-subscriber asymmetry (one publisher, 1000 subscribers), the publisher's synaptome fills entirely with subscriber edges (via incoming promotion). It loses connectivity to peers outside the subscriber set.
- A Byzantine attacker can spam a target node with incoming connections (fake publishes, fake subscribe-acks), causing the target to promote the attacker into its synaptome and learn to route toward it.
- Incoming synapses lack the inertia lock that LTP-reinforced synapses get. A promoted incoming synapse can be evicted immediately on the next capacity crunch, creating jitter.

Measurement needed:

- Asymmetric pub/sub test: one publisher, 100 subscribers. Measure whether the publisher's synaptome remains diverse or becomes subscriber-dominated.
- Incoming promotion ablation: disable incoming promotion and re-run all benchmarks. Measure the latency/success degradation.
- Incoming inertia variant: when an incoming synapse is promoted, give it an inertia lock (same as LTP). Measure whether this improves or hurts the system.
- Byzantine resistance: add a 1% "spammer" attack where adversary nodes spam incoming connections to a target. Measure whether the target's routing degrades.

Why it matters: Incoming promotion is elegant (passive learning of who's interested in you), but it's a one-way signal. It doesn't encode "do I need to reach this peer" — only "this peer reached me." In a heterogeneous network, that asymmetry can be exploited.

10.8 Parameter Sensitivity and Tuning Brittleness

The gap: NH-1 claims 12 parameters vs NX-17's 44. But the existing materials don't measure how sensitive the protocol is to variations in those 12.

What breaks:

- If `INERTIA_DURATION` is too short (< 10 epochs), recently-learned synapses evict too fast and learning doesn't stick.
- If `RECENCY_HALF_LIFE` is too long (> 100 epochs), stale synapses linger and block capacity.
- If `DECAY_GAMMA` is too close to 1 (> 0.998), weights don't decay and the protocol converges to the initial synaptome.
- If `DECAY_GAMMA` is too low (< 0.990), weights decay too fast and every lookup feels like a cold start.

The existing materials don't show sensitivity curves for any of these.

Measurement needed:

- One-factor-at-a-time (OFAT) sensitivity: vary each of the 12 parameters $\pm 20\%$ around the default and measure global latency / pub/sub delivery.
- Interaction study: measure whether changes in one parameter (e.g., `INERTIA_DURATION`) make other parameters (e.g., `RECENCY_HALF_LIFE`) more or less sensitive.

- Scale-dependent tuning: does the optimal parameter set change with N? Run sensitivity at 1K, 10K, 50K nodes.
- Heterogeneous-network tuning: measure whether the 12 parameters that work for "15% highway" also work for "0% highway" and "100% highway."

Why it matters: The consolidation from 44 → 12 parameters is advertised as a feature. But if those 12 are brittle (high sensitivity, strong interactions), the "easier to tune" claim collapses. This is a pre-deployment audit.

10.9 Temporal Pub/Sub Cache Semantics Under Partition

The gap: The bounded replay cache on every relay is designed to recover from transient churn. But under a network partition, a relay in the West can accumulate messages the East never sees, and vice versa.

What breaks:

- After partition heals, subscribers in the East re-subscribe to a relay in the West. The West relay's cache has messages from while East was partitioned. Should it replay all of them, or only recent ones?
- If it replays all, the subscriber gets a flood of "stale" messages and has to filter them by timestamp.
- If it replays only recent ones, the subscriber misses history.
- A malicious relay can selectively replay — holding back some messages, forwarding others — to poison the subscriber's view of history.

Measurement needed:

- Partition healing test: partition the network for 100 lookups (~10 seconds at 10 msg/sec publish rate). Each partition accumulates messages independently. On partition heal, measure what subscribers see: do they get history from both sides, one side, or neither?
- Cache overflow test: if a relay's cache is full and a new message arrives during partition isolation, which message is dropped? Is the policy FIFO, LRU, or by topic?
- Timestamp validation: add a validation check that replayed messages' publish timestamps are strictly monotonic per topic. Measure how often this check fails under partition.

Why it matters: The temporal pub/sub mechanism is elegant under stable conditions. Under partition, it becomes a consistency problem. This is the Byzantine gap the existing red team flags.

10.10 Deployment Realism: The Gap Between Simulator and Real WebRTC

The gap: The simulator models "WebRTC with 50–100 connection cap" as a realistic constraint. But real WebRTC adds:

- ICE gathering (1–3 s to find candidate paths)
- STUN/TURN (latency spikes, asymmetry)
- DTLS handshake (RTT + crypto overhead)
- Media stream setup (if the connection is ever used for media)
- Browser garbage collection (spikes every 100–500 ms)
- Tab backgrounding (network I/O suspended, timer resolution drops to 1 s)

What breaks:

- A node that tries to add 10 new synapses in parallel will fire 10 concurrent ICE gathers, consuming all the browser's network threads. Other lookups stall.
- A GC pause during a critical lookup can cause an RPC timeout (3 s) and trigger premature dead-peer eviction.

- Incoming synapses from backgrounded tabs may silently fail to upgrade to full connections, leaving the synaptome with "connections" that aren't actually live.

Measurement needed:

- Concurrent-connection-setup test: measure the cost of firing N new ICE gathers in parallel. What's the crossover point where sequential setup (slower but serialized) beats parallel setup (faster individually but contends for resources)?
- GC pause modeling: inject a 50 ms GC pause every 200 ms and measure lookup success degradation.
- Backgrounding test: run the network, background 10% of nodes (timers drop to 1 s resolution), and measure pub/sub delivery latency and success.
- Battery drain: estimate energy cost of keeping 50–100 concurrent connections alive (periodic keep-alives, socket I/O, radio on). Is the energy budget compatible with a mobile deployment?

Why it matters: The existing red team mentions "frictionless connection fantasy" but doesn't quantify the friction. Phase 1 of the action plan is friction modeling — this is the specifics.

10.11 Tier Summary

Tier 1 (Deploy Blockers)

1. **Latency noise / jitter** (10.4) — AP routing depends on clean RTT signals. Real networks have $\pm 30\%$ variance. Measure EMA constant sensitivity and jitter-tolerance.
2. **Bandwidth saturation** (10.6) — Relay nodes will overflow under Zipf-distributed pub/sub. Need load-aware routing and admission control.
3. **Concurrent connection setup cost** (10.10) — ICE gathering is the real bottleneck, not the synaptome logic. Measure how many parallel setup operations the browser can sustain.

Tier 2 (Correctness Under Stress) 4. **Convergence under heterogeneous churn** (10.1) — Ghinita-Teo adaptive annealing (temperature as function of local failure rate) is needed. 5. **Asymmetric reachability / partition healing** (10.3) — Temporal consistency of replay cache and loop detection need specification. 6. **Incoming synapse bias** (10.7) — Incoming promotion is elegant but one-directional; needs either asymmetric-learned inertia or explicit load-aware gating.

Tier 3 (Security Hardening) 7. **S2 cell eclipse and Sybil swarms** (10.5) — Integrate Vivaldi RTT validation or proof-of-location to prevent prefix forgery. 8. **Byzantine relay hijacking** (10.5, 10.7) — Add MaxDisjoint replication of axon roots (Harvesf-Blough 2007). 9. **Parameter sensitivity** (10.8) — OFAT sweep at 1K / 10K / 50K nodes; verify $\pm 20\%$ changes don't break the network. 10. **Temporal cache under partition** (10.9) — Specify replay semantics for ragged partitions; add timestamp validation.

These ten items, plus the five already in existing materials, form a complete pre-deployment audit. Tier 1 should block launch. Tier 2 should be solved before wide deployment. Tier 3 can be post-launch hardening.

12. Operational Framework — Parameter Tuning

The consolidation from NX-17's 44 parameters to NH-1's 12 is only valuable if those 12 can be tuned by operators without deep protocol knowledge.

11.1 The Twelve Parameters

Parameter	Domain	Default	Range	What it controls
-----------	--------	---------	-------	------------------

INERTIA_DURATION	LEARN	20 epochs	5–50	How long a freshly-reinforced synapse is eviction-proof
RECENCY_HALF_LIFE	FORGET	50 epochs	20–200	How fast unused synapses become evictable
DECAY_GAMMA	FORGET	0.995	0.980–0.999	Base weight decay rate per epoch
T_INIT	EXPLORE	1.0	0.5–2.0	Initial exploration temperature
T_MIN	EXPLORE	0.05	0.01–0.20	Minimum exploration floor
T_COOLING	EXPLORE	0.9997	0.9990–0.9999	Temperature decay per lookup
EPSILON	EXPLORE	0.05	0.01–0.20	Probability of random first hop
LOOKAHEAD_ALPHA	NAVIGATE	5	2–10	Candidates probed per 2-hop evaluation
MAX_SYNAPTOME_SIZE	STRUCTURE	50	30–100	Connection capacity per node
HIGHWAY_PCT	STRUCTURE	15	0–50	Fraction of nodes with 256-synapse cap
GEO_BITS	STRUCTURE	8	0–16	S2 cell prefix width (0 = no geography)
MAX_HOPS	NAVIGATE	40	10–100	Safety cap on lookup depth

11.2 Three Levels of Tuning

Level 1: Operator Dials (Non-Expert)

For a deployment operator with no protocol knowledge:

Dial 1: "How churn-prone is my network?"

LOW (< 5% / day) → T_COOLING = 0.99995, DECAY_GAMMA = 0.998

MEDIUM (5–20%) → defaults (0.9997, 0.995)

HIGH (> 20%) → T_COOLING = 0.9990, DECAY_GAMMA = 0.990

Dial 2: "What's my device class mix?"

All browser → MAX_SYNAPTOME = 50, HIGHWAY_PCT = 0

Mixed (typical) → MAX_SYNAPTOME = 50, HIGHWAY_PCT = 15

Server-heavy → MAX_SYNAPTOME = 256, HIGHWAY_PCT = 50

Dial 3: "How latency-sensitive is my app?"

Message queue → LOOKAHEAD_ALPHA = 2, T_MIN = 0.10 (faster, less exploration)

Real-time streaming → LOOKAHEAD_ALPHA = 8, T_MIN = 0.05 (more exploration for resilience)

Interactive chat → defaults (balance)

Dial 4: "What geographic span?"

Single region → GEO_BITS = 12 (fine-grained locality)

Continental	→ GEO_BITS = 8 (default)
Global	→ GEO_BITS = 4 (coarse-grained)

These four dials map onto the 12 parameters via a lookup table. An operator never sees `DECAY_GAMMA` directly — they see "how churn-prone."

Level 2: Tuner Parameters (Protocol Engineer)

A protocol engineer deploying a new network sweeps the 12 parameters against a baseline workload:

1. **Baseline measurement:** Run the network for 1 hour under "standard load" (500 lookups/min, 5% churn, 50% pub/sub). Record p50/p95/p99 lookup latency, pub/sub delivery rate and latency, synaptome diversity, temperature distribution.
2. **OFAT sweep:** Vary each parameter $\pm 20\%$ around default. For each variant, re-run baseline and record deltas.
3. **Interaction audit:** For any parameter pair that showed sensitivity, run a 2D grid sweep to measure interaction.
4. **Scale sensitivity:** Run the OFAT at $N \in \{1K, 10K, 50K\}$ and measure whether optimal parameters shift with scale.
5. **Decision rule:** Adopt a new parameter value only if p50 latency improves $\geq 5\%$ OR pub/sub delivery improves $\geq 2\%$, *and* p99 latency doesn't regress $> 10\%$.

Level 3: Research Parameters (R&D)

Beyond the 12 core parameters, second-order knobs for research:

- **Vitality exponent:** Currently $\text{vitality} = \text{weight} \times \text{recency}$. Could be $\text{weight}^\alpha \times \text{recency}^\beta$.
- **Latency penalty form:** Currently $\frac{1}{2}^{(\text{latency_ms}/100)}$. Could be exponential, sigmoid, step function.
- **Incoming promotion threshold:** Currently after 2 uses. Could be adaptive based on synaptome load.
- **Annealing reheat amount:** Currently spikes to 0.5. Could be function of dead-peer count or local churn rate.
- **Triadic closure frequency:** Currently fires on every transit. Could be gated by minimum co-appearance count.

These are not exposed to operators — they're for protocol improvement cycles.

11.3 Tuning Workflow

Week 0–1: Pre-launch Tuning

1. Run baseline measurements at target scale (e.g., 25K nodes).
2. Measure actual churn rate, latency distribution, traffic pattern.
3. Map deployment profile to one of the four operator dials.
4. Run baseline workload with dial's recommended parameters for 24 hours.
5. If p99 latency is acceptable, proceed. If not, escalate to Level 2 tuning.

Week 1–2: Launch Deploy with tuned parameters. Instrument the live network to collect per-node and aggregate metrics (Section 12).

Week 2–4: Adaptation If metrics drift (churn increases, latency increases): recalculate which dial applies; if dial changes, apply the new Level 1 parameters and monitor; if dial stays the same but metrics drift, escalate to Level 2 on the live network with canary rollout.

Month 1–3: Steady State Collect operational data. Every 2 weeks, measure whether current parameters still fit deployment or whether scale/churn/workload changes warrant re-tuning.

13. Observability and Alerting

12.1 Per-Node Metrics (Tier 1)

Every node continuously tracks and periodically reports (every 60s):

Synaptome Health

- Size (current / max)
- Stratum coverage (% of 64 strata with ≥ 1 peer)
- Age distribution (% synapses < 100 epochs old, 100–1000, > 1000)
- Weight distribution (p10/p50/p90 of weights)
- Inertia lock status (# currently locked, % of synaptome)

Routing Performance

- Lookup success rate (% completed vs. hit MAX_HOPS)
- Hops per lookup (p50 / p95 / p99)
- Latency per lookup (p50 / p95 / p99 in ms)
- Two-hop lookahead frequency
- Iterative fallback frequency
- Dead-peer eviction rate

Learning Dynamics

- LTP fire rate (per 100 lookups)
- Triadic closure rate
- Hop caching rate
- Incoming promotion rate
- Annealing replacement rate

Exploration State

- Current temperature T
- Temperature age (epochs since last reheat)
- Epsilon-greedy fire rate
- Reheat frequency

Network Churn

- Dead-peer discoveries per 100 lookups
- Inbound connection attempts (successes / failures)
- Inbound connection churn (# closed per 100 epochs)
- Synaptome stability (% unchanged over last 1000 lookups)

12.2 Network-Level Aggregates (Tier 2)

Roll up per-node metrics. Report every 5 minutes:

Routing Convergence

- Global p50 / p95 / p99 lookup latency
- Lookup success rate across all nodes

- Stratum coverage distribution (% of nodes with > 80% coverage)
- Weight distribution (global p50/p90)
- Synaptome diversity (entropy of stratum distribution per node, averaged)

Learning Rate

- Global LTP fire rate (% of lookups generating reinforcement)
- Global triadic closure rate
- Global annealing replacement rate
- Learning concentration (% of learning happening in top 10% of nodes)

Churn Adaptation

- Network dead-peer rate
- Reheat frequency distribution (p50/p95 per node)
- Recovery time post-churn (lookups until latency returns to baseline)

Anomaly Indicators

- Synaptome age skew (% of nodes with median synapse age > 10K epochs)
- Temperature stuck (% of nodes with T unchanged for > 10K epochs)
- Stratum blackout (any stratum group with < 1 peer per node on average)

12.3 Pub/Sub Metrics (Tier 3)

Track every axonal tree separately:

Per-Topic Metrics

- Subscriber count (current, trend)
- Publisher rate (messages/sec)
- Delivery latency (p50 / p95 / p99 from publish to subscriber receipt)
- Delivery success rate (% received by all subscribers)
- Dropped message rate
- Replay cache hit rate
- Axon tree depth (max, p95)
- Axon branching factor

Tree Integrity

- Dead-children per tree
- Orphans per tree
- Root stability (# times root changed per 1000 publishes)
- Re-subscribe success rate

Load Distribution

- Relay load concentration (Gini coefficient)
- Relay saturation (# relays with TX queue > 80%)
- Fan-out breadth per relay

12.4 Operator Dashboards (Tier 4 — The Four Dials)

Abstract raw metrics into the four operator dials:

Dial 1: Churn Health

INPUT: Dead-peer rate, reheat frequency, synaptome stability

OUTPUT: "LOW" / "MEDIUM" / "HIGH"

ACTION: If drift from expected, suggest retuning T_COOLING and DECAY_GAMMA

Dial 2: Device Class Balance

INPUT: Synaptome size distribution, inbound connection success rate

OUTPUT: "All browser" / "Mixed" / "Server-heavy"

ACTION: If < 80% of server-class nodes are at capacity, suggest increasing HIGHWAY_PCT

Dial 3: Latency Profile

INPUT: p99 lookup latency, two-hop lookahead frequency, epsilon-greedy rate

OUTPUT: "Message queue" / "Interactive" / "Real-time"

ACTION: If p99 regresses, suggest increasing LOOKAHEAD_ALPHA

Dial 4: Geographic Span

INPUT: Regional latency ratio (500 km / global), stratum distribution

OUTPUT: "Single region" / "Continental" / "Global"

ACTION: If regional latency exceeds expected, suggest increasing GEO_BITS

12.5 Alerting Rules

Critical Alerts (Page Immediately)

1. **Network Partition Detection** — Any stratum group with zero peers across > 50% of nodes
2. **Lookup Success Collapse** — Network-wide success rate < 95%
3. **Pub/Sub Delivery Failure** — Any topic with delivery rate < 90% for > 5 minutes
4. **Synaptome Collapse** — > 20% of nodes with synaptome size < 30

Warning Alerts (Investigate Within Hours)

5. **Latency Creep** — p95 lookup latency increases > 20% over 1 hour
6. **Learning Stall** — LTP fire rate drops below 10% of lookups for > 30 minutes
7. **Stratum Imbalance** — > 30% of nodes with stratum coverage < 50%
8. **Relay Concentration** — Any pub/sub topic with Gini coefficient > 0.75

Informational Alerts (Log for Analysis)

9. **Parameter Drift** — Any node with parameters diverging from network baseline
10. **Byzantine Suspicion** — Any node with synaptome > 80% toward a single peer
11. **Cache Overflow** — Any pub/sub replay cache running at > 90% capacity
12. **Temperature Stuck** — Any node with temperature unchanged for > 10K epochs

14. Diagnosis Playbooks

13.1 Playbook: Lookup Success Collapse (Critical Alert #2)

Symptoms: Network-wide lookup success < 95%

Investigation Steps:

1. **Check stratum coverage distribution**
 - o If > 20% of nodes have stratum coverage < 50%: network partitioned or lost long-range connectivity. Increase GEO_BITS by 4 to force re-seeding.

- If stratum coverage is normal: routing algorithm failure, not topology failure. Check two-hop lookahead fire rate.

2. Check dead-peer eviction rate

- If > 30 evictions per 100 lookups: high churn or aggressive decay. Increase INERTIA_DURATION 20 → 30; decrease DECAY_GAMMA 0.995 → 0.998.
- If < 5 evictions per 100 lookups: low churn but routing failing — likely Byzantine or parameter misconfiguration. Check temperature distribution.

3. Check pub/sub separately

- If pub/sub > 95% but lookups < 95%: pub/sub axonal tree is live but random lookups failing. Increase LOOKAHEAD_ALPHA 5 → 8.

4. Check for Byzantine attack

- Sample 10 nodes with low success rates. Inspect their synaptomes: do they have > 50% weight to a single peer? If yes: probable incoming-synapse spam. Implement IP-based rate limiting on incoming connections from that peer.

Recovery Steps:

A. **Soft recovery (no reboot):** Increase EPSILON 0.05 → 0.15 for 1 hour. Spike all node temperatures to 0.5. Run 100 lookups per node with increased LOOKAHEAD_ALPHA. Measure: does success recover to > 98%?

B. **Hard recovery (reboot required):** Roll back last parameter change. Rebuild synaptomes from scratch with stratified bootstrap. Increase INERTIA_DURATION to 30 during warmup. Run 10K lookups to re-learn routes.

13.2 Playbook: Pub/Sub Delivery Failure (Critical Alert #3)

Symptoms: Topic delivery rate < 90% for > 5 minutes

Investigation Steps:

1. Check relay node health

- Sample the root relay for this topic. Is it alive? If dead: re-subscribe should find new root within 10s. Check re-subscribe success rate; if < 90%, increase frequency 10s → 5s.
- If alive: check load. TX queue backed up? Relay is saturated. Consider hot-root migration.

2. Check tree integrity

- Dead-children > 10% of subscriber count: subscribers detaching faster than re-subscribing fixes it. Increase re-subscribe frequency 10s → 3s.
- Orphans > 5%: subscribers re-subscribing but not finding any relay in time. Check stratum distribution of relays; may need more relays in subscriber cells.

3. Check cache behavior

- Replay cache hit rate < 50%: subscribers missing history. Increase cache size 100 → 200 or extend retention.

4. Check for Byzantine relay

- Sample delivered vs. dropped messages. Pattern-based dropout (always drops when load > 50%) suggests Byzantine relay. Temporarily blacklist; re-route topic to alternate.

Recovery Steps:

A. **Immediate (within 1 minute):** Increase re-subscribe frequency to 3s. Increase replay cache to 200 messages.

B. **Short-term (1–5 minutes):** If relay saturated, trigger hot-root migration: select next-best relay (least loaded, same cell). Or spawn secondary relay in different cell (MaxDisjoint placement).

C. **Long-term (> 5 minutes):** Analyze root-cause logs. If Byzantine: implement reputation penalties. If load: implement load-aware relay selection. If churn: increase GEO_BITS to stabilize relay discovery.

13.3 Playbook: Latency Creep (Warning Alert #5)

Symptoms: p95 lookup latency increases > 20% over 1 hour

Investigation Steps:

1. Check for hot topics (Zipf-driven load)

- Query pub/sub metrics; is one topic getting > 50% of publishes? If yes: that topic's relays are bottlenecks. Monitor relay queue depth; consider load-aware routing.

2. Check synaptome churn

- LTP fire rate dropping (fewer lookups succeeding with short paths)? Old shortcuts evicting faster than new ones forming. Increase INERTIA_DURATION; decrease DECAY_GAMMA.

3. Check temperature distribution

- Most nodes cold ($T \approx T_{\text{MIN}}$)? Network annealed and stopped exploring. Increase T_{MIN} 0.05 $\rightarrow$ 0.10 to keep some exploration active. Or spike all temperatures to 0.3 for 1 hour to force re-exploration.

4. Check for network saturation

- Aggregate throughput trending upward? Latency creep under load is normal. Monitor whether creep stabilizes.

5. Check two-hop lookahead effectiveness

- Lookahead fire rate > 70%: algorithm struggling to find decisive first hops. Increase LOOKAHEAD_ALPHA 5 $\rightarrow$ 8. Check if stratum coverage degraded.

Recovery Steps:

A. **Soft recovery (minutes):** Increase LOOKAHEAD_ALPHA 5 $\rightarrow$ 7 temporarily. Monitor p95; if improvement > 5%, keep change.

B. **Targeted recovery (hours):** If hot topic identified: implement load-aware relay selection. AP scoring: $AP_{*} = \max(0.5, 1 - \text{relay_load} / \text{saturation_cap})$. Test on canary (10% of nodes) for 1 hour.

C. **Structural recovery (days):** Analyze synaptome stability. If > 20% turnover per hour: INERTIA_DURATION too short. Increase 20 $\rightarrow$ 30 epochs.

15. Production Readiness Checklist

Before deploying NH-1 to production, verify:

Infrastructure

- ☐ Logging pipeline can handle per-node metrics from every node every 60s
- ☐ Time synchronization across all nodes within 100 ms (for latency attribution)
- ☐ Alerting system can ingest Tier 1 and 2 metrics and page on-call within 30s
- ☐ Runbooks (playbooks 13.1–13.3) are accessible to on-call and walked through in drills

Operational Readiness

- ☐ On-call trained on the four operator dials; can shift parameter recommendations within 5 minutes
- ☐ Canary rollout process defined (10% → 50% → 100%) and tested with a parameter change
- ☐ Rollback procedure documented (how to rebuild synaptomes if a parameter change is bad)
- ☐ Dashboard live and on-call has practiced reading it under simulated alerts

Protocol Verification

- ☐ Simulator-to-production gap analysis complete (jitter, timeouts, bandwidth caps modeled)
- ☐ OFAT sensitivity analysis done; parameters not brittle ($\pm 20\%$ change doesn't break the network)
- ☐ Byzantine resistance test passed (1% spammer nodes don't degrade network to $< 95\%$ success)
- ☐ Partition healing verified (Slice World equivalent test on libp2p)

Measurement

- ☐ Baseline metrics captured from pre-production testnet (what does "healthy" look like?)
- ☐ Anomaly detection thresholds calibrated (alert #5: what counts as "creep"?)
- ☐ Tier 3 (pub/sub) metrics collection end-to-end tested

Documentation

- ☐ Protocol specification complete and matches implementation
- ☐ Operator dial mappings documented (churn profile → DECAY_GAMMA/T_COOLING conversions)
- ☐ Failure modes catalogued (what does "stratum blackout" mean? what causes it?)
- ☐ Runbooks written and reviewed by operations team

16. Reputation and Byzantine Resistance

All the playbooks above assume nodes are honest but fallible. NH-1 has no built-in Byzantine resilience.

15.1 What Needs to Be Added

1. Per-synapse reputation score

Add a `reputation ∈ [0, 1]` field to each `Synapse` :

- Starts at 0.5 (neutral)
- Increments on successful relay
- Decrements on relay failure
- Used in AP scoring: `AP *= reputation` (low-reputation peers penalized)

2. Incoming-synapse rate limiting

An attacker can spam incoming connections to exhaust your inbound cap:

- Track inbound arrival rate per source peer
- If > 10 incoming syn/sec from one peer, rate-limit subsequent ones
- Each rate-limited connection decrements reputation

3. Replay cache poisoning detection

An attacker can relay false messages on a pub/sub topic:

- Add `contentHash` to every published message
- Subscribers verify: if relay sends a message with wrong `contentHash`, mark relay Byzantine
- Relay gets reputation decrement; topic root switches away if reputation drops below 0.3

4. Triadic closure gating

An attacker can introduce itself into every triadic closure:

- Track which peers introduce you to others (transit partners)
- Only trust introductions from peers with reputation > 0.7
- Or: require 2+ independent paths through different peers before forming a triadic edge

Implementation size: ~200 lines added to NH-1 (reputation tracking + AP multiplier + rate limiting).

15.2 Verification Test

Run the network with 1% Byzantine relays. Measure whether:

- Lookup success stays > 95% (reputation penalties route around attackers)
- Pub/sub delivery stays > 90% (axonal tree heals around Byzantine relays)
- Reputation scores converge (honest peers develop high scores, attackers low scores)

This is Phase 3 of the red-team action plan; it's not blocking but it's necessary before real deployment.

17. Deployment Timeline

Status (v0.3.53). The initial production Axona stack — `axona-peer` and `axona-bridge`, running the NH-1 implementation — is live (see §9.6). The plan below describes the path from this initial deployment to a multi-thousand-node operating network; it predates the v0.3.51 production launch and is retained as a forward-looking calibration for the scale-up phase.

Month 1: Simulator Hardening

- Complete OFAT sensitivity analysis (red-team item 10.8)
- Run all ten additional red-team tests
- Validate Phase 1 (friction modeling) preliminary implementation
- Publish revised deck with sensitivity curves and operator dials

Month 2: yz.p2pnetwork Integration

- Port NH-1 to libp2p (NeuromorphicDHT.js → TypeScript in yz.p2pnetwork)
- Implement Tier 1 metrics collection
- Build operator dashboard (the four dials + critical alerts)
- Test concurrent connection setup on real WebRTC

Month 3: Testnet (Small Scale)

- Deploy to 100-node testnet (Docker containers, simulated churn)
- Run baseline measurement (1 hour at standard load)
- Verify all Tier 1 metrics working and alerting
- Run playbooks with simulated failures
- Collect operator feedback on dashboard usability

Month 4: Testnet (Medium Scale)

- Scale to 1K nodes
- Run OFAT sensitivity analysis on real hardware
- Calibrate alert thresholds
- Run Byzantine resistance test (1% spammer nodes)
- Verify partition healing

Month 5: Staging (Production-Like)

- Deploy to 5K-node staging environment (geographically distributed)
- Run 1-week soak test
- Measure operator on-call experience
- Implement reputation system if not already done
- Finalize runbooks based on staging experience

Month 6: Limited Production Release

- Deploy with 500 initial nodes
- Run under 10× normal expected load (stress test)
- Monitor all Tier 1, 2, 3 metrics continuously
- If latency, success rate, or pub/sub delivery degrade > 5%: roll back and diagnose
- If stable for 1 week: proceed to 2K nodes

Month 7–8: Production Ramp

- Scale 2K → 5K → 10K nodes over 2 weeks
- At each step, pause for 1 week of observation
- If any metric diverges from testnet baseline, investigate before scaling further

Month 9+: Production at Scale

- Monitor continuously; dial tuning as needed
- Measure real-world workload distribution (Zipf?)
- Implement load-aware relay selection if hot-topic concentration appears
- Plan Phase 2 (friction modeling with real latency, timeouts, jitter)

16.1 Scale Transitions

The parameters tuned for 5K nodes may not be optimal at 50K.

At 10K nodes (2× current):

- p50 latency will increase slightly
- Pub/sub tree depth will increase
- Monitor: do stratum coverage remain constant?
- If coverage drops: increase GEO_BITS

At 25K nodes (5× current):

- Synaptome diversity becomes critical
- Annealing replacement rate may drop
- If LTP fire rate and annealing both drop: decrease DECAY_GAMMA

At 50K nodes (10× current):

- Browser-class nodes start to struggle
- Increase HIGHWAY_PCT 15% → 25%
- Or increase MAX_SYNAPTOME_SIZE 50 → 75

At 100K+ nodes:

- Consider two-tier topology: browser nodes route to nearest server-class node
- Outside NH-1's current design scope; may need architectural change

18. Comparison to Prior Art

17.1 Coral DSHT

Coral (Freedman, Freudenthal, Mazières — NSDI 2004) is a **distributed sloppy hash table** that powered Coral CDN. Hierarchical RTT clusters: each node measures RTT and joins multiple nested clusters. Lookup is local-first.

Aspect	Coral DSHT	N-DHT
Structure	Multiple nested DHTs, one per RTT cluster	Single flat synaptome with weighted edges
Locality discovery	Active RTT measurement at join + cluster membership	Passive — observed traffic reinforces useful edges; S2 prefix seeds
Storage semantics	Sloppy — multiple replicas per key, return-first-found	Strict — one canonical XOR-closest node per key
Lookup	Local cluster first, escalate outward	Single greedy AP walk over weighted synaptome
Adaptation	Cluster boundaries are static thresholds	Continuous: weights update on every successful path
Designed for	Read-heavy content distribution	Routing + pub/sub — dynamic membership, real-time delivery
Pub/sub support	Out of scope	First-class via axonal trees

Coral's design choice was: "give up the canonical mapping to win locality." NH-1's design choice was: "keep the canonical mapping; make locality emerge from learning." Different engineering trade-offs against the same core problem.

17.2 Vivaldi

Vivaldi (Dabek, Cox, Kaashoek, Morris — SIGCOMM 2004) is a **decentralized network coordinate system**. Each node continuously adjusts a synthetic position vector so that Euclidean distance approximates measured RTT.

Aspect	Vivaldi	N-DHT
Mechanism	Synthetic coordinates in N-D Euclidean space	Hebbian reinforcement on routing edges
What's learned	Position vector per node that predicts RTT	Weight per synapse, reinforced by traffic
Output	RTT prediction (any pair)	Ranked list of next-hops (per lookup)
Locality discovery	Emergent from RTT measurements	Imposed via S2 cell ID prefix
Convergence guarantee	Yes — coordinate descent on RTT residuals	Empirical — depends on traffic mixing
Pub/sub support	Out of scope	First-class — axonal tree built on routed synaptome

A Vivaldi-style coordinate system is a candidate for replacing NH-1's structural S2 prefix with a learned locality primitive. Future benchmarks may include a Vivaldi-style protocol for completeness — both as comparison reference and forward-looking direction.

17.3 Route-Diversity DHTs

Castro et al. (OSDI 2002) is the foundational paper on Byzantine fault tolerance in DHTs. Three jointly-necessary mechanisms: constrained routing tables, secure node ID assignment, redundant routing.

Harvesf & Blough (IEEE P2P 2007) makes redundant routing concrete: replicate at $(n+1) \times B^m$ locations to produce d disjoint routes. **MaxDisjoint replica placement + Neighbor Set Routing**: 90% lookup success with half the network compromised.

Aspect	Castro / Harvesf-Blough	N-DHT
Threat model	Byzantine — malicious nodes that lie about routing	Crash-failure — honest peers that disappear
Mechanism	Replicate target at d disjoint placements	Maintain weighted synaptome with overlapping candidates
What's redundant	Multiple disjoint paths to same key	Multiple weighted next-hop options per lookup
Activation	Always — every lookup queries replicas in parallel	Reactive — iterative fallback only when greedy AP dead-ends
Cost	$\times d$ storage, parallel network load per lookup	One synaptome; lookup load unchanged

Their work delivers strong Byzantine resilience (90% success at 50% malicious) at the cost of $\times d$ storage and parallel query load. N-DHT delivers strong crash-failure resilience (100% delivery under 5% churn) at the cost of zero extra storage. Combining them — MaxDisjoint replication of NH-1 axon-tree roots — is the obvious next step for Byzantine-tolerant pub/sub.

17.4 Hotspot-Aware Placement

Makris, Tserpes, Anagnostopoulos (IEEE BIGDATA 2017) addresses what consistent-hashing DHTs ignore: **request rates are not uniform even when keys are**. Real workloads follow Zipf's law. On a 24-node Redis cluster: hottest node received 222K requests, others ~1–10K. Response time on hotspot was 5× cluster median.

Mechanism: Directory For Exceptions (DFE). Hybrid placement keeping consistent hashing as default with a small distributed override.

For NH-1: when an axon-tree root's request rate exceeds a permissible threshold, migrate the topic to a less-loaded peer in the synaptome and install a DFE-style redirect at the original location. Combined with secondary geo / RTT-based splitting, closes the gateway concentration failure mode under Zipf-popular topics.

19. Future Work

Three phases of next-step work, ordered from highest behavioral impact to security hardening.

18.1 Phase 1 — Friction Modeling (Highest Priority)

What the simulator is missing:

- **CONNECTION_SETUP_MS = 1500–2000 ms** — new synapses sit in `PENDING` and are excluded from AP scoring until setup elapses
- **RPC_TIMEOUT_MS = 3000 ms** — sends to silently-dropped nodes stall before iterative fallback or next AP hop is tried
- **Request/reply RPC** — refactor `routeMessage` to trace forward path *and* reverse path. Either failure fails the RPC; reply may take a different path back

18.2 Phase 2 — Load & Noise Dynamics

Make the protocol measurable under stress:

- **Load-dependent latency.** $\text{effective_delay} = \text{base_delay} \times (1 + \text{active_msg_rate} / \text{bandwidth_cap})$. Saturating gateways get penalized in AP scoring.
- **Bandwidth dropping.** When a node exceeds its modeled cap, drop incoming messages instead of forwarding.
- **Jitter injection.** Add `Normal(0, JITTER_SIGMA)` to per-hop RTT. Verify LTP EMA doesn't oscillate.
- **Zipf-distributed publish workload.** Stress NH-1 with skewed publish rates to expose whether single-root axon trees saturate.
- **Adaptive anneal/reheat driven by observed churn rate** (Ghinita & Teo 2006). Each node maintains rolling estimate of local failure rate μ and join rate λ . Anneal cooling rate, reheat amount, and synapse staleness threshold all become functions of (μ, λ) rather than fixed constants.
- **Liveness vs accuracy decoupled.** Today `_evictAndReplace` (liveness) and `_tryAnneal` (accuracy) fire as mixed side-effects. Splitting them into independent threshold-triggered channels (Ghinita-Teo) gives operators one dial per channel.
- **Variable-churn benchmark.** Add peak-hour test: alternate 3/sec churn for 30 min with 0.5/sec for 90 min, repeat. Measures NH-1's adaptation time at each transition.

18.3 Phase 3 — Structural Integrity & Trustless Locality

- **Bidirectional eviction agency.** When A connects to B, B independently runs its own stratified eviction to decide whether to keep the reverse edge.

- **Vivaldi RTT integration.** Replace self-declared S2 prefix with organically learned coordinates — Sybil-resistant locality without trusting peers' self-claims.
- **Geographic proof of work / IP-ASN binding.** Require geo-prefix to align with node's actual ASN/region or carry a hash-cash stamp.
- **Topic replication via MaxDisjoint placement** (Harvesf & Blough 2007). Replicate every axon-tree root at d disjoint locations.
- **Hot-axon-root migration** (Makris et al. 2017). When axon-tree root's request rate exceeds threshold, migrate the topic to a less-loaded peer.
- **Target-QoS knob** (Ghinita & Teo 2006 framing). Expose `targetLookupFailureRate` and `targetMedianLatency` as user-facing parameters. The system self-tunes underlying constants to hit them. Precondition for a metaplastic NH-1.

18.4 The Unfinished Consolidation

NH-1 unified admission into a single vitality gate, but conceptual rules remain that could be further simplified:

- **Stratified bootstrap vs. random bootstrap** — Why not let annealing *discover* stratum diversity rather than enforcing it at init?
- **Two-hop lookahead as a separate operation** — Could this merge into AP scoring as a cost-of-uncertainty term?
- **Temperature reheat on dead-peer discovery** — Could this be replaced by a learned "confidence" signal?

These are not flaws — they're the remaining 18% of "why NH-1 is slower than NX-17." Each rule earned its keep. But they hint at a potential **NH-2**: further consolidation where those three merge into a unified confidence-and-exploration model.

18.5 Bottom Line

The brain is production-ready. The body is not — but the body is now the next *measurable, scopeable* problem rather than a tangle of interacting mechanisms. Each phase above produces falsifiable measurements; the simulator becomes the lab bench for the next iteration.

20. References

Whitepaper — `documents/Neuromorphic-DHT-Architecture.md` (companion repository, v0.67) **Source**
+ data — `github.com/YZ-social/dht-sim`

Architecture

- Saltzer, Reed, Clark · *End-to-End Arguments in System Design* (ACM TOCS 1984) — function placement principle
- Clark · *The Design Philosophy of the DARPA Internet Protocols* (SIGCOMM 1988)

Foundational DHT work

- Maymounkov & Mazières · *Kademlia: A Peer-to-peer Information System Based on the XOR Metric* (IPTPS 2002)
- Rowstron & Druschel · *Pastry: Scalable, decentralized object location and routing* (Middleware 2001)
- Zhao et al. · *Tapestry: A Resilient Global-Scale Overlay* (IEEE J-SAC 2004)
- Stoica et al. · *Chord: A Scalable Peer-to-peer Lookup Service* (SIGCOMM 2001)
- Ratnasamy et al. · *A Scalable Content-Addressable Network* (SIGCOMM 2001)

Latency-aware DHTs

- Dabek, Li, Sit, Robertson, Kaashoek, Morris · **Designing a DHT for low latency and high throughput** (NSDI 2004) — DHash++; the **36 floor** analysis (§ 4.3) anchors the absolute-latency reference
- Freedman, Mazières · *Sloppy Hashing and Self-Organizing Clusters* (IPTPS 2003) — the Coral DSHT design
- Freedman, Freudenthal, Mazières · **Democratizing Content Publication with Coral** (NSDI 2004) — Coral CDN deployment
- Gummadi et al. · *The Impact of DHT Routing Geometry on Resilience and Proximity* (SIGCOMM 2003)

Adaptive coordinates

- Dabek, Cox, Kaashoek, Morris · **Vivaldi: A Decentralized Network Coordinate System** (SIGCOMM 2004)
- Cox, Dabek, Kaashoek, Li, Morris · *Practical, Distributed Network Coordinates* (HotNets 2003)
- Ledlie, Gardner, Seltzer · *Network Coordinates in the Wild* (NSDI 2007)

Byzantine resistance / route diversity

- Castro, Druschel, Ganesh, Rowstron, Wallach · **Secure Routing for Structured Peer-to-Peer Overlay Networks** (OSDI 2002) — foundational Byzantine-DHT paper; **constrained routing + secure ID assignment + redundant routing** triplet
- Harvesf, Blough · **The Design and Evaluation of Techniques for Route Diversity in Distributed Hash Tables** (IEEE P2P 2007) — **MaxDisjoint replica placement** + Neighbor Set Routing; 90% lookup success at 50% node failure
- Baumgart & Mies · *S/Kademlia: A Practicable Approach Towards Secure Key-Based Routing* (ICPADS 2007)

Load balancing / hotspot mitigation

- Karger, Lehman, Leighton, Panigrahy, Levine, Lewin · *Consistent Hashing and Random Trees* (STOC 1997)
- Rao, Lakshminarayanan, Surana, Karp, Stoica · *Load balancing in structured P2P systems* (IPTPS 2003)
- Makris, Tserpes, Anagnostopoulos · **A novel object placement protocol for minimizing the average response time of get operations in distributed key-value stores** (IEEE BIGDATA 2017) — **Directory For Exceptions (DFE)** + threshold-triggered migration

Adaptive maintenance / churn handling

- Mahajan, Castro, Rowstron · *Controlling the Cost of Reliability in Peer-to-peer Overlays* (IPTPS 2003)
- Krishnamurthy, El-Ansary, Aurell, Haridi · *A statistical theory of Chord under churn* (IPTPS 2005)
- Ghinita, Teo · **An Adaptive Stabilization Framework for Distributed Hash Tables** (IPDPS 2006) — local statistical estimation of (μ, λ, N) + threshold-triggered liveness/accuracy checks

Pub/Sub

- Castro, Druschel, Kermarrec, Rowstron · *SCRIBE: A Large-Scale and Decentralized Application-Level Multicast Infrastructure* (JSAC 2002)

Substrate (learning, decay, pruning)

- Hebb · *The Organization of Behavior* (1949) — synaptic potentiation

- Bliss & Lømo · *Long-lasting Potentiation of Synaptic Transmission in the Dentate Area of the Anaesthetized Rabbit* (J. Physiology 1973)
- Ebbinghaus · *Über das Gedächtnis* (1885) — exponential forgetting curve
- Frey & Morris · *Synaptic tagging and the late phase of LTP* (Nature 1997) — biological analog of weight × recency retention
- Kirkpatrick, Gelatt, Vecchi · *Optimization by Simulated Annealing* (Science 1983)
- LeCun, Denker, Solla · *Optimal Brain Damage* (NeurIPS 1989) — prune lowest-magnitude connections
- O'Neil, O'Neil, Weikum · *The LRU-K Page Replacement Algorithm* (SIGMOD 1993) — multi-history recency for cache replacement
- Watts & Strogatz · *Collective Dynamics of Small-World Networks* (Nature 1998)
- Google · *S2 Geometry Library* (2011) — <https://s2geometry.io/>
- Hilbert · *Ueber die stetige Abbildung einer Line auf ein Flächenstück* (Mathematische Annalen 1891)

End of whitepaper. Total length ≈ 80 pages typeset.

Suggested next steps for the reader:

1. *If you're a researcher interested in the mechanism: Sections 3–5 are the architectural core.*
2. *If you're an operator preparing for deployment: Sections 11–14 are the operational core.*
3. *If you're a security researcher: Sections 9–10 and 15 are the threat model.*
4. *If you're a protocol contributor: Sections 6, 8, and 18 frame what's worth working on next.*